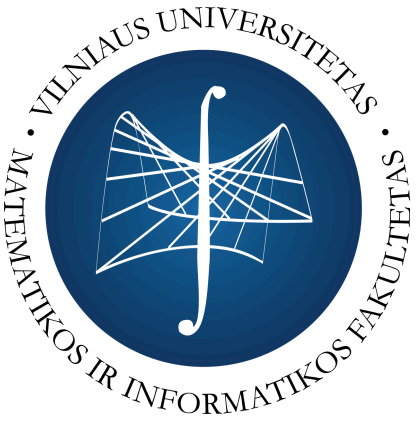


VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
PROGRAMŲ SISTEMŲ STUDIJŲ PROGRAMA

PS Software Design
LAB 1

Kristina Noreikaitė
Emilija Gegužytė
Ugnė Rupšytė
Ieva Survilaitė

Vilnius
2026

Table of contents

1. Introduction....4
    1.1 Overview....4
    1.2 Target users....4
    1.3 Definitions....5
    1.4 Scope....5
2. Feature list / Business requirements....6
    2.1 Identity and access....6
    2.2 Messaging....6
    2.3 Files....7
    2.4 Calls....8
    2.5 Presence....8
    2.6 Workflow....8
    2.7 Notifications and search....9
3. Business flows....10
        BF-01 Register an account....10
        BF-02 Sign in and reset a forgotten password....11
        BF-03 Send a message....14
        BF-04 Create a channel and add members....15
        BF-05 Share a file....16
        BF-06 Work on a channel workflow....18
4. Non-Functional Requirements....20
    4.1 Usage patterns and reference load....20
        4.1.1 Usage patterns....20
        4.1.2 Reference load....20
        4.1.3 Where data is stored....21
    4.2 Performance....22
    4.3 Scalability....23
    4.4 Availability and reliability....24
    4.5 Security and privacy....24
    4.6 Usability and accessibility....25
    4.7 Compatibility and portability....26
    4.8 Maintainability, testability and observability....26
    4.9 System limits and data retention....26
5. High level architecture....27
    5.1 Architectural overview....27
    5.2 System structure and deployment....28
    5.3 Bounded contexts....29
    5.4 Package Structure....31

5.4.1. Server Packages..... 31
5.4.2 Client packages..... 32
5.5 Design patterns and key mechanisms..... 33
5.5.1 Presence state machine..... 34
5.5.2 Workflow state machine..... 35
5.6 Class diagrams..... 36
5.6.1 Identity & Access and Messaging..... 36
5.6.2 Application Services and Ports..... 37
5.6.3 Presence and Workflow..... 39
5.6.4 Domain Events and Notifications..... 40
5.6.5 Files..... 41
5.7 Runtime Views..... 42
5.7.1 Sending a Message..... 42
6. UI/UX wireframes..... 44
7. Data model..... 56
7.1. User, OneTimeCode, RefreshToken, AuditEntry..... 56
7.2. Message, Conversation, Membership, MessageAttachment, File..... 58
7.3. Workflow, Notification..... 59
8. API contract..... 60
8.1 Overview & Conventions..... 60
8.2 Structure of the Attached Contract (openapi.yaml)..... 61

1. Introduction

TeamChat is a real-time team communication platform for one organization. It is similar to platforms like Microsoft Teams or Slack. Team members can send instant messages in channels and direct messages, share files and see who from the team is online. Channels can also carry lightweight tasks (workflows) that move through stages: To Do, In Progress, and Done.

1.1 Overview

Teams often split their work across a chat tool, a video tool, shared drives and a separate task board. Context gets lost between them, and people cannot see whether a colleague is available or already working on a task. TeamChat puts these activities into one browser application:

- Talk: public and private channels for topics and teams, and direct messages between two users. Messages appear for everyone instantly.
- Share: files uploaded to a conversation are stored once in the file storage. The message carries a link to the file and only members of that conversation can open it.
- See availability: online, away or offline status for every user, updated automatically.
- Track work: a workflow attached to a channel, assigned to a member who can mark it "On It" and move it through its stages.
- Stay informed: notifications about new messages, mentions and workflow changes on the main page.

1.2 Target users

User	Description	Main goals
Team member	Any registered user of the organisation	Communicate with colleagues, share files, work on assigned workflows
Channel owner	The member who created a channel	Manage channel members and settings, create and assign workflows
Administrator	A user responsible for the organisation's workspace	Manage users, remove inappropriate content

1.3 Definitions

Channel - A named conversation for a group of members; public (anyone in the organisation can join) or private (by invitation).
DM - A direct message conversation between two users.
Conversation - A channel or a DM.
File storage - The place where uploaded files are kept. A message contains a link to a stored file, not the file itself.
Presence - A user's availability status: Online, Away or Offline.
Workflow - A task attached to a channel, with a description, an assignee and a stage.
Stage - The state of a workflow: To Do, In Progress or Done.

1.4 Scope

In scope: registration and sign-in with email confirmation, channels and DMs, real-time messaging, file sharing, presence, workflows, notifications, search and personal data export.

Out of scope: several workspaces or organisations per deployment, native mobile applications and a responsive mobile layout, integrations with external tools, calls.

2. Feature list / Business requirements

The platform has 41 functional requirements. Each one has an ID and a MoSCoW priority. M = Must, S = Should, C = Could, W = Won't (for now).

2.1 Identity and access

ID	Requirement	Priority
FR - 01	A user can create an account with email and password, confirmed with a 6-digit code sent by email.	M
FR - 02	A user can sign in with email and password and sign out.	M
FR - 03	A user who forgot their password can reset it through a code sent by email.	M
FR - 04	A user can edit their profile: display name and profile picture.	S
FR - 05	A user can change their password while signed in.	S
FR - 06	A user can export their data and delete their account.	S
FR - 07	An administrator can deactivate a user account.	C
FR - 08	Sign-in with external providers (Google, Microsoft).	W

2.2 Messaging

ID	Requirement	Priority
FR - 09	A user can create a public or private channel with a unique name and an optional description.	M

FR - 10	A user can browse and join public channels.	M
FR - 11	A user can join a private channel only by invitation from a channel member.	M
FR - 12	A user can leave a channel.	M
FR - 13	The channel owner can add and remove channel members.	M
FR - 14	A user can start a DM with any other user of the organisation.	M
FR - 15	A member can send a text message to a channel or DM, it appears to all online members in real time.	M
FR - 16	A user sees the message history of a conversation, newest first, and can load older messages.	M
FR - 17	A user sees which conversations have unread messages.	S
FR - 18	A user can edit and delete their own messages, an administrator can delete any message.	S
FR - 19	A user can mention another member with ‘@name’, which notifies them.	S
FR - 20	A user can react to a message with an emoji.	C
FR - 21	A user can reply to a message in a thread.	C
FR - 22	Typing indicator ("Anna is typing…").	C

2.3 Files

ID	Requirement	Priority
FR - 23	A member can upload files to a conversation (up to 10 files per message, 25 MB each). Each file is stored once in file storage, and the message contains a link to it.	M
FR - 24	Members of the conversation can download a shared file, non-members cannot.	M
FR - 25	A user can see all files shared in their conversations on the Files page.	S
FR - 26	Images are shown as a preview in the conversation.	C

2.4 Calls

ID	Requirement	Priority
FR - 27	A user can start an audio call from a DM, the other user receives an incoming call and can accept or decline.	C
FR - 28	A participant can turn the video and microphone on and off and leave the call.	C
FR - 29	A member can start a call in a channel and up to 4 members can join it.	W
FR - 30	Screen sharing during a call.	W
FR - 31	Call recording.	W

Calls are out of scope for the first version. They are listed here for a possible later version.

2.5 Presence

ID	Requirement	Priority
FR - 32	Every user's status (Online, Away, Offline) is shown next to their name and updated automatically.	M
FR - 33	A user can set their status manually (e.g. Away).	S

2.6 Workflow

ID	Requirement	Priority
FR - 34	A channel member can create a workflow in the channel with a title and description.	M
FR - 35	A workflow can be assigned to a channel member, or a member can claim it with "On It".	M
FR - 36	The assignee can move the workflow through the stages To Do - In Progress - Done.	M
FR - 37	A user sees all workflows assigned to them on the Workflows page.	S

FR - 38	A workflow can have a due date.	C

2.7 Notifications and search

ID	Requirement	Priority
FR - 39	A user receives notifications for new DMs, mentions, channel invitations and workflow changes. The latest are shown on the main page	M
FR - 40	A user can search messages and file names in the conversations they belong to.	S
FR - 41	Email notifications for unread messages while offline.	C

3. Business flows

The six flows below cover the Must features of Section 2. Each flow names the screens of Section 6 it uses and the requirements it implements.

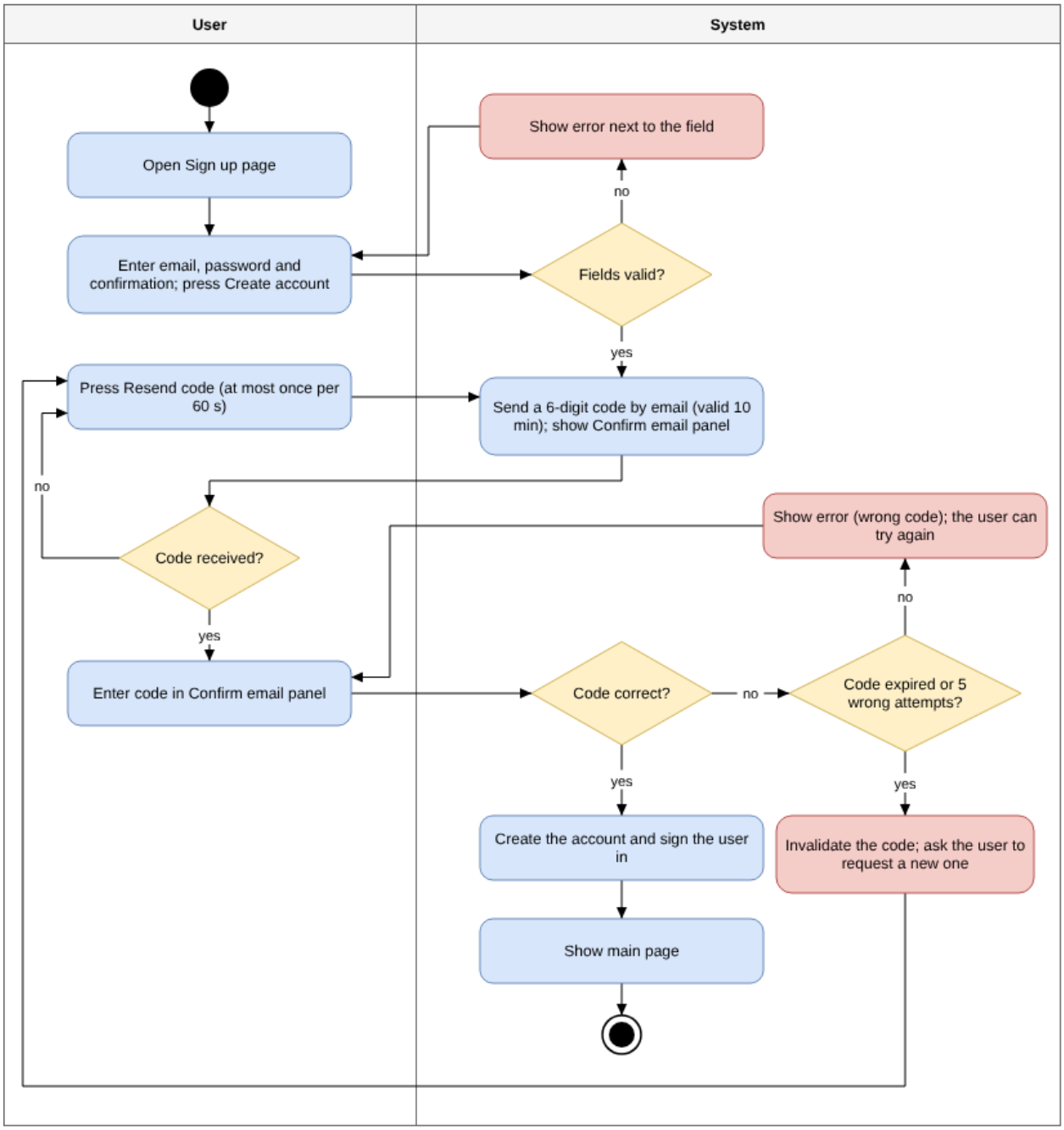
BF-01 Register an account

Screens: Sign up - Confirm email - Main page.

Requirements: FR-01, SEC-01, SEC-04.

1. The user opens the Sign up page and enters email, password and password confirmation.
2. The system validates the fields. An invalid email, a weak password or non-matching passwords are shown next to the field (USE-03).
3. The user presses Create account. The system sends a 6-digit code to the email and opens the Confirm email panel.
4. The user enters the code.
    1. Correct code: the account is created, the user is signed in and redirected to the main page.
    2. Wrong or expired code: an error is shown. After 5 wrong attempts the code becomes invalid and the user requests a new one with Resend code.
    3. No code received: the user presses Resend code (at most once per 60 s).

BF-01 Register an account



BF-01 activity diagram: sign-up form - validation - code sent - code check (correct / wrong or expired / resend) - main page.

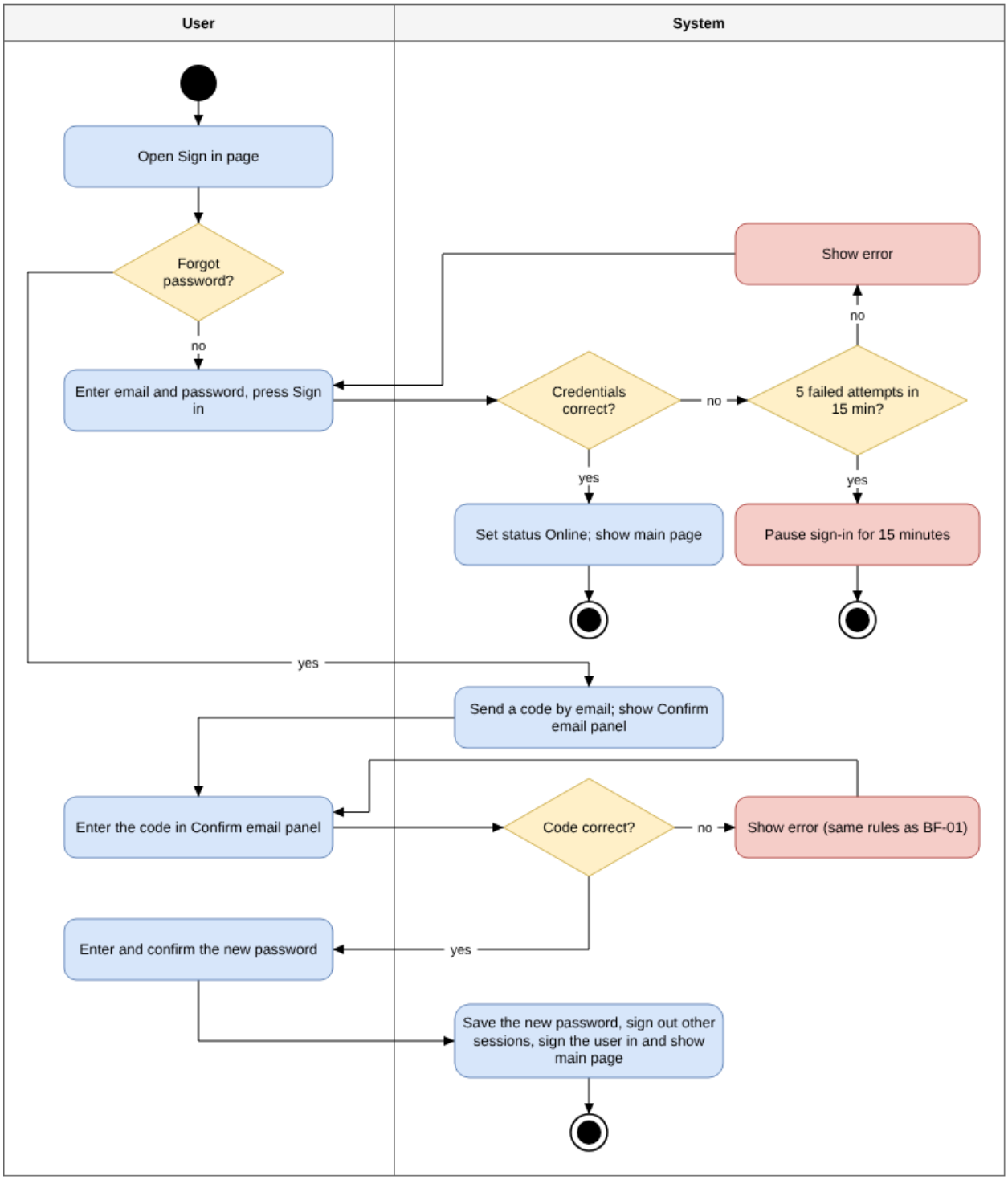
BF-02 Sign in and reset a forgotten password

Screens: Sign in - Main page, or Sign in - Confirm email - Change password - Main page.

Requirements: FR-02, FR-03, SEC-03, SEC-07.

1. The user enters email and password and presses Sign in.
    1. Correct: the user is redirected to the main page and their status becomes Online.
    2. Wrong: an error is shown. After 5 failed attempts sign-in is paused for 15 minutes.
2. If the password is forgotten, the user presses Forgot password?. The system sends a code to the email.
3. The user enters the code in the Confirm email panel (same rules as BF-01, step 4).
4. The user enters and confirms a new password and presses Sign in. All other sessions of the user are signed out.

BF-02 Sign in and reset a forgotten password



BF-02 activity diagram: sign in (correct / wrong / paused after 5 attempts) and the forgot-password branch with code check and new password.

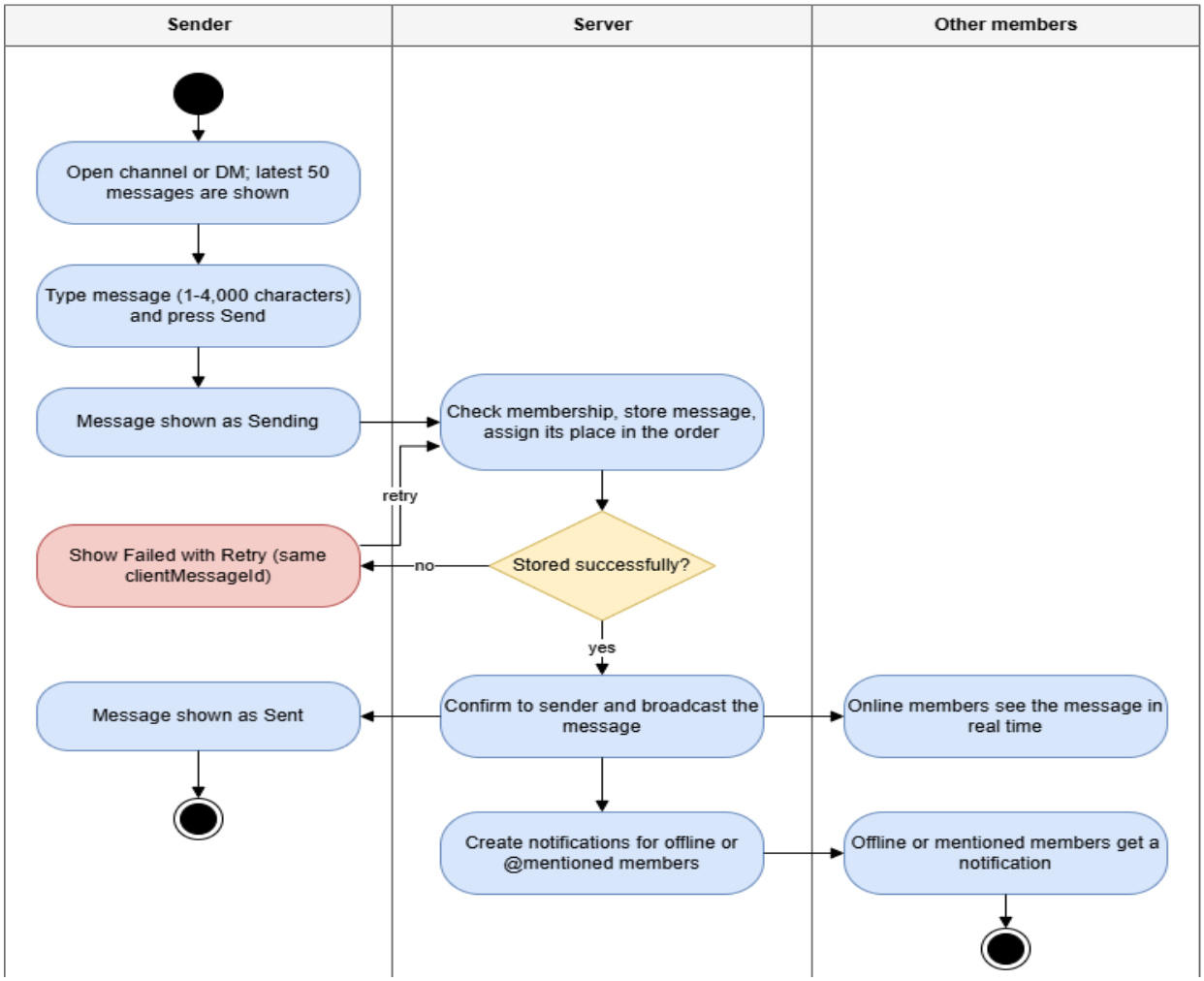
BF-03 Send a message

Screens: Main page - Channel or DM view.

Requirements: FR-15, FR-16, FR-19, REL-02–REL-04, USE-02.

1. The user opens a channel or DM from the left panel. The latest 50 messages are shown.
2. The user types a message (1–4,000 characters) and presses Send. The message is shown at once with the status Sending.
3. The system checks that the user is a member of the conversation, stores the message and gives it its place in the conversation order.
4. The sender sees the status Sent. All online members receive the message in real time.
5. Members who are offline, or who were mentioned with @name, get a notification.
6. If sending fails, the message shows Failed with a Retry action. A retry never creates a duplicate.

BF-03 Send a message



BF-03 activity diagram with swimlanes Sender / Server / Other members: Sending - membership check - stored - Sent - real-time delivery and notifications, failure - Failed - Retry.

BF-04 Create a channel and add members

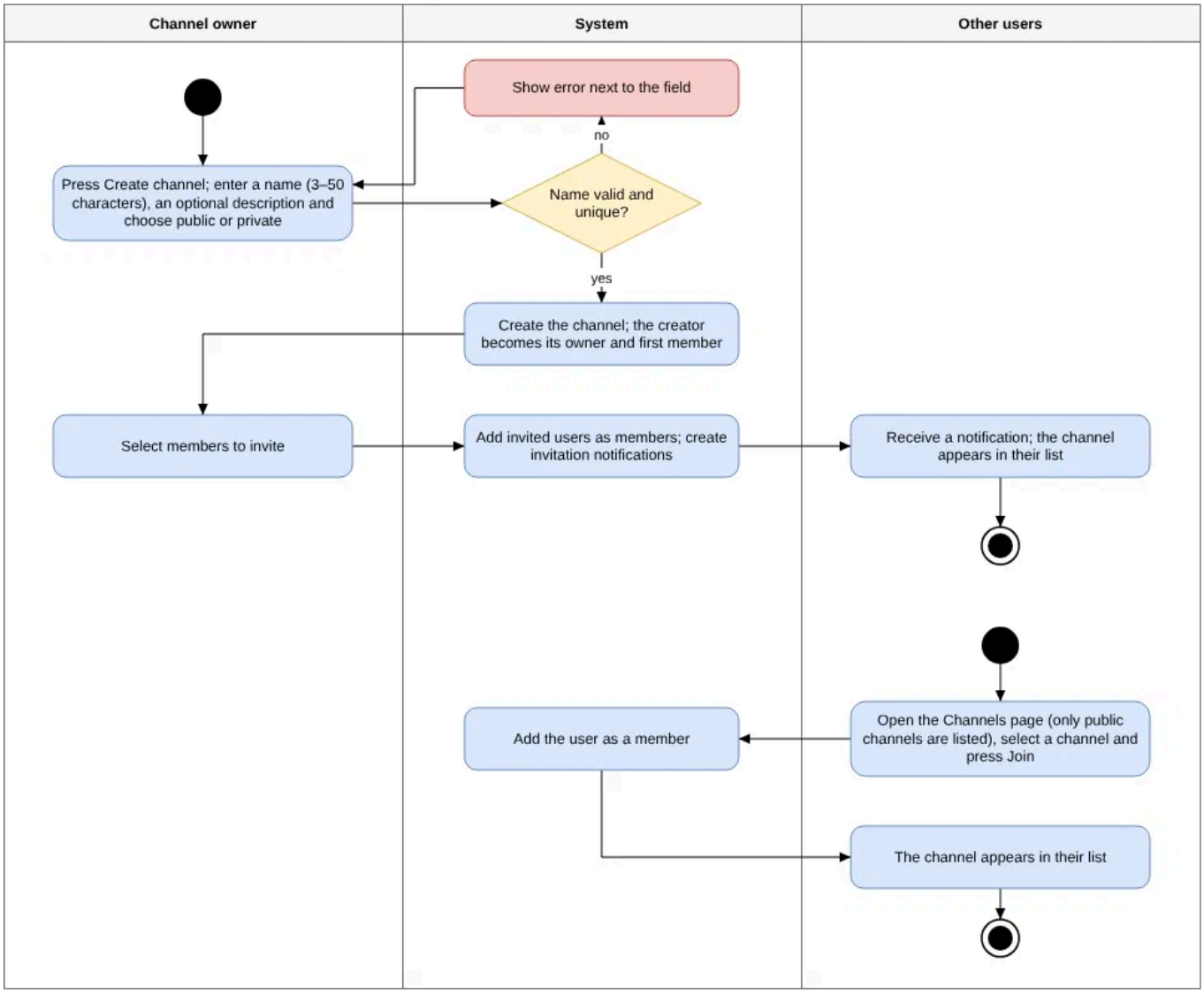
Screens: Channels page - Channel view.

Requirements: FR-09–FR-13.

1. The user presses Create channel, enters a unique name (3–50 characters), an optional description and chooses public or private.
2. The system creates the channel; the creator becomes its owner and first member.

3. The owner invites members. Each invited user gets a notification and the channel appears in their list.
4. Other users can find a public channel on the Channels page and join it themselves.

BF-04 Create a channel and add members



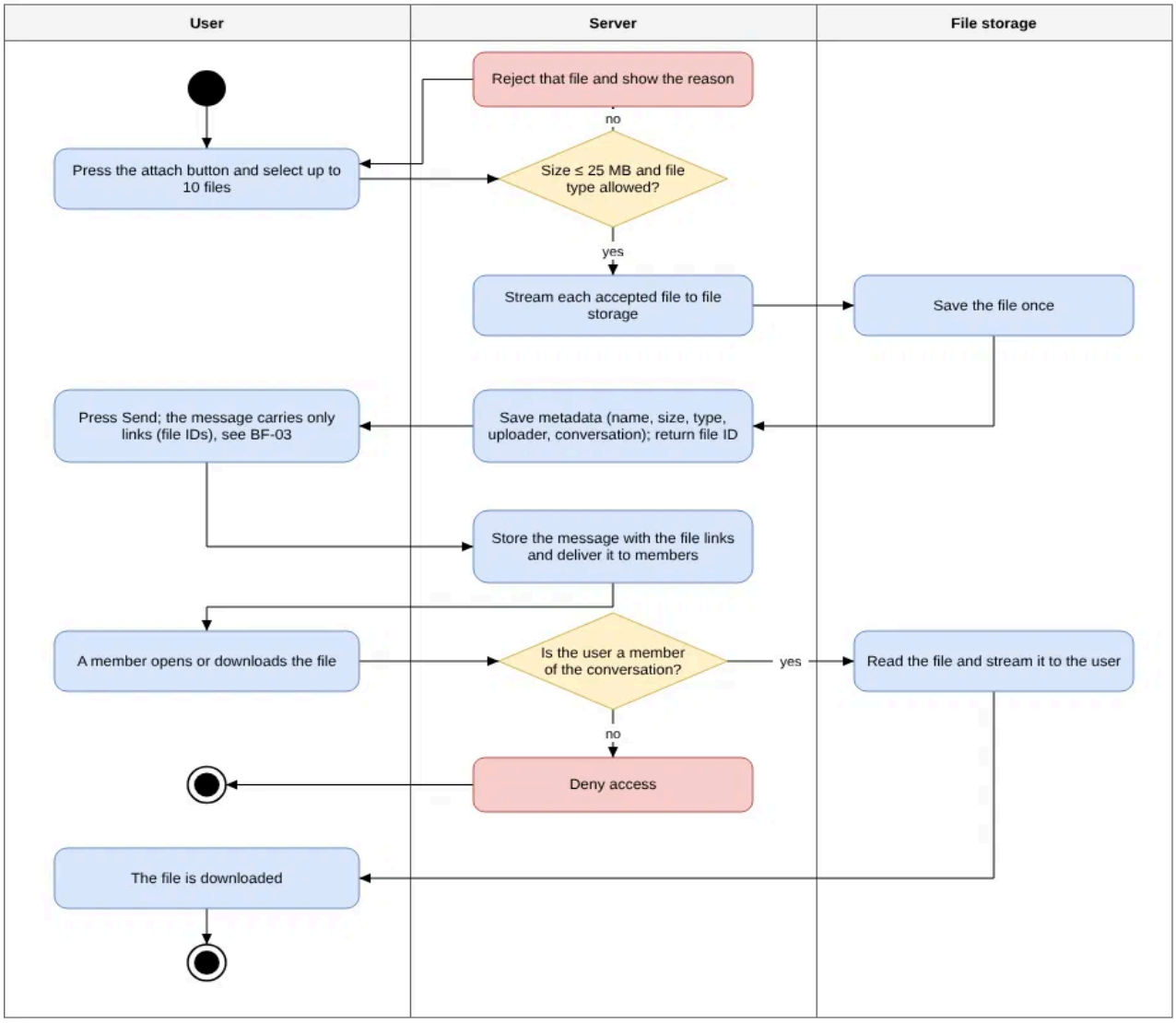
BF-05 Share a file

Screens: Channel or DM view - Files page.

Requirements: FR-23–FR-25, SEC-08.

1. In a conversation the user presses the attach button and selects up to 10 files.
2. The system checks size (25 MB) and file type. A rejected file is shown with the reason; the others are uploaded.
3. Each accepted file is uploaded once to file storage. The system saves its metadata (name, size, type, uploader, conversation) and returns a file ID.
4. The user sends the message (BF-03). The message contains only links to the stored files, not the files themselves, so the message size does not depend on file size or the number of members.
5. Members see the file name, size and a download link in the conversation and on the Files page. A file is transferred only when a member opens or downloads it.
6. When a download is requested, the system checks that the user is a member of the conversation. Non-members are denied access.

BF-05 Share a file



BF-06 Work on a channel workflow

Screens: Channel view - Create workflow - Workflows page.

Requirements: FR-34–FR-37, FR-39.

1. A channel member creates a workflow with a title and description (1–2,000 characters). It starts in the stage To Do.
2. The creator assigns it to a member, or any member claims it with On It. The assignee gets a notification.

3. The assignee moves it to In Progress when work starts and to Done when it is finished.
4. Every stage change is shown in the channel and notifies the creator. The assignee sees the workflow on the Workflows page.



BF-06 state diagram of a workflow: Created - To Do (assigned or claimed with On It) - In Progress - Done.

4. Non-Functional Requirements

This section lists the quality attributes the system must satisfy. Every requirement has an ID, a measurable acceptance criterion and a MoSCoW priority (M = Must, S = Should, C = Could), so the implementing team knows what to build and how to check it. All load figures are first derived from expected usage patterns (4.1.1) and only then turned into limits (4.1.2).

4.1 Usage patterns and reference load

4.1.1 Usage patterns

TeamChat is deployed for one organisation (Section 1.4). The figures below describe how that organisation is expected to use the system. They are design assumptions, and every limit in this section is calculated from them.

- **Organisation**: a mid-size company in one time zone, working day 08:00–18:00 (10 h), Monday–Friday. Almost all traffic falls inside a 10 h window, 250 days a year.
- **Registered users**: every employee has one account, up to 1,000 accounts.
- **Daily active users**: about 60 % of accounts sign in on a working day, about 600 users.
- **Connection time**: an active user keeps TeamChat open about 5 h of the 10 h day, so on average 600 × 5 / 10 = 300 users are connected; at peak (morning, after lunch) about 400.
- **Messages**: an active user sends about 40 messages per day, 600 × 40 = 24,000 messages per day.
- **Message rate**: 24,000 messages over 10 h is 0.7 msg/s on average; the busiest hour carries 3× the average (2 msg/s); short bursts (start of day, replies to an announcement) reach 5× the busiest hour (10 msg/s).
- **Recipients**: an average channel has about 30 members, half of them online, so one message is pushed to about 15 WebSocket connections.
- **Files**: about 300 files are shared per working day, 1.5 MB on average, about 450 MB per day and 110 GB per year.

4.1.2 Reference load

All targets in 4.2–4.9 are defined for the following reference deployment. Each limit is the usage pattern above plus a safety margin.

- **Tenancy:** one organisation per deployment (a single shared workspace). Multi-workspace

support is out of scope (1.4).

- **Users:** up to 1,000 registered users and up to 400 connected at the same time (peak from 4.1.1). The limit exists because the reference deployment has a single application server and a single database server; a larger organisation needs a second application server (SCAL-01, SCAL-02).
- **Traffic:** up to 20 messages per second system-wide, which is 2× the expected burst of 10 msg/s. With about 15 recipients per message this is about 300 real-time events per second.
- **Burst case:** if all 400 connected users press Send within the same second, the system must accept all 400 messages without loss or duplicates (PERF-08). This is a short peak, not a sustained rate: the per-user limit of 30 messages per minute (SEC-07) caps the sustained maximum at 400 × 0.5 = 200 msg/s.
- **Data volume:** up to 6,000,000 stored messages (one year at 24,000 per day) and up to 500 GB of stored files (1,000 users × 500 MB quota). The expected file volume after one year is about 110 GB.
- **Client:** desktop browser, layout as in Section 6. Native mobile apps, a responsive mobile layout and audio/video calls are out of scope (1.4).
- **Deployment:** one application server, one database server (PostgreSQL) and one file volume, delivered as containers (COMP-02). Email is sent through an external SMTP service.

4.1.3 Where data is stored

- **Users, conversations, memberships, workflows, notifications, audit log, codes and tokens:** relational database (PostgreSQL) on the database server; below 1 GB after one year; kept as described in 4.9.
- **Messages:** relational database, table message; about 6,000,000 rows and 3 GB after one year; at most 4,000 characters per message.
- **File metadata** (name, type, size, owner): relational database, table file; about 75,000 rows after one year.
- **File contents:** file volume on the application server under a random storage_key (SEC-08); about 110 GB after one year; 25 MB per file and 500 MB per user.
- **Search index:** full-text index inside the relational database, derived from messages and file names; about 1 GB; rebuilt from messages if lost.

- **Presence and WebSocket connections:** memory of the application server only, never persisted (Section 7); a few MB; lost on restart and rebuilt when clients reconnect.
- **Backups:** separate backup storage with daily copies of the database and file volume (REL-07); 7 daily copies.

4.2 Performance

The platform must provide responsive interactions, fast message delivery and efficient access to stored information under the reference load (4.1.2).

- **PERF-01 Message delivery (M)**: p95 ≤ 500 ms between the server accepting a message and the event arriving at an online recipient, at 20 msg/s and 400 connected users.
- **PERF-02 API response time (M)**: p95 ≤ 300 ms for read requests and ≤ 600 ms for write requests (file transfer excluded).
- **PERF-03 History loading (M)**: the first page of 50 messages is shown within 1 s; older messages load with cursor pagination, 50 per page.
- **PERF-04 Presence propagation (S)**: a presence change is visible to other users within 2 s.
- **PERF-05 Search performance (S)**: the first page of message search results is returned within 2 s with 6,000,000 stored messages.
- **PERF-06 Client start-up (S)**: after sign-in the main page is interactive within 3 s on a 10 Mbit/s connection.
- **PERF-07 File transfer (M)**: uploads and downloads are streamed in 64 KB chunks, so server memory per transfer stays at one 64 KB buffer whatever the file size. Test: 10 parallel 25 MB uploads and 10 parallel downloads raise server memory use by less than 50 MB.
- **PERF-08 Message burst (S)**: a burst of 400 messages from 400 different users within 1 s is fully accepted: every message is stored exactly once, p95 delivery during the burst ≤ 3 s, and PERF-01 holds again within 10 s after the burst.

How file streaming works (PERF-07). A file is never held in server memory as a whole; it passes through a small buffer from the network to the disk and back.

1. Upload. The client sends the file as multipart/form-data (Section 8, Files API). The server reads the file part as an input stream, 64 KB at a time, and passes the stream to FileStorage.save(key, stream), which writes each chunk to a temporary file on the file volume.

2. Checks while reading. The first chunk is used to detect the real content type from the file's first bytes (SEC-08); a type not on the allow-list stops the upload at once. A byte counter stops the upload with HTTP 413 when it passes 25 MB or the user's remaining quota (4.9).
3. Finish or clean up. After the last chunk the temporary file is renamed to its random storage_key and one file row with the metadata is inserted. If the connection breaks or a check fails, the temporary file is deleted and no row is written.
4. Download. After the membership check (SEC-05), FileStorage.open(key) returns an input stream that is copied to the HTTP response 64 KB at a time, with Content-Length, Content-Type and Content-Disposition: attachment. HTTP Range requests are supported, so a broken download can resume.

What happens in the burst case (PERF-08). Each message is saved in one short database transaction that increments Conversation.last_seq (Section 7). Messages to different conversations are written in parallel through the database connection pool. Messages to the same conversation wait for each other on that row, which keeps the order (REL-04); at about 5 ms per transaction, 400 messages in one channel take about 2 s. If the server's request queue is full, it answers HTTP 503 with Retry-After; the client keeps the message in the Sending state and retries with the same clientMessageId, so nothing is lost or duplicated (REL-03).

4.3 Scalability

The architecture must support the reference load and allow selected infrastructure components to be replaced or extended without major changes to domain logic.

- **SCAL-01 Stateless API (S):** no per-user state is kept in server memory except the WebSocket connection registry and presence; authentication uses tokens, not server sessions. Together with SCAL-02, two application server instances behind a load balancer work correctly.
- **SCAL-02 Replaceable event distribution (C):** real-time fan-out goes through the EventBus and RealtimeBroadcaster interfaces; the in-process implementation can be replaced by a broker (e.g. Redis pub/sub) without changing domain code.
- **SCAL-03 Data growth (M):** required indexes exist: messages by (conversation_id, seq), notifications by (recipient_id, created_at), memberships by (user_id), files by (uploader_id). PERF targets still hold with 6,000,000 messages.
- **SCAL-04 Headroom (C):** a load test at 2× the reference load (40 msg/s, 800 connected

users) finishes without errors, with p95 delivery ≤ 1 s.

4.4 Availability and reliability

The platform must remain operational, protect confirmed messages from loss and recover from connection failures and infrastructure problems.

- **REL-01 Availability (S):** 99.5 % per month for the reference deployment, planned maintenance excluded.
- **REL-02 No message loss (M):** once the server confirms a message to the sender, it is persisted and survives a server restart.
- **REL-03 Idempotent sending (M):** the client generates a clientMessageId; repeating a request with the same id returns the original message and never creates a duplicate.
- **REL-04 Ordering (M):** all participants see the messages of a conversation in the same order, defined by the server-assigned per-conversation sequence number seq (Section 7).
- **REL-05 Reconnection (M):** after a lost connection the client reconnects automatically with exponential backoff (1 s up to 30 s), then fetches missed messages and notifications through REST using its last-seen cursor.
- **REL-06 Fault isolation (S):** a failure of the SMTP service or the file volume does not stop text messaging; the UI shows an error only for the affected feature.
- **REL-07 Backup (S):** daily backup of the database and the file volume, kept 7 days; RPO 24 h, RTO 4 h.

4.5 Security and privacy

The platform must protect user credentials, communications, files and personal information against unauthorised access and common security threats.

- **SEC-01 Password storage (M):** Argon2id or bcrypt hashes; passwords are never stored or logged in plain text; minimum 8 characters with at least one letter and one digit.
- **SEC-02 Transport security (M):** HTTPS and WSS (TLS 1.2 or higher) everywhere.
- **SEC-03 Tokens (M):** access token lifetime ≤ 15 min; refresh token 7 days, rotated on every use and revoked on sign-out or password change. The WebSocket is authenticated during the handshake with a short-lived, single-use ticket obtained through REST (Section 8.1).
- **SEC-04 One-time codes (M):** 6-digit codes for email confirmation and password reset; valid 10 min; at most 5 wrong attempts; resend at most once per 60 s; stored hashed.

- **SEC-05 Authorisation (M):** every REST request and every WebSocket subscription checks membership. Non-members never receive content of a private channel or DM. A file is available only to members of a conversation that has a message with this file attached (Section 7.2).
- **SEC-06 Input handling (M):** server-side validation of all input, parameterised queries, escaping of message text against XSS, CSRF protection if cookies are used.
- **SEC-07 Rate limiting (S):** at most 5 failed sign-ins per 15 min per account and IP; at most 30 messages per minute per user; excess requests get HTTP 429.
- **SEC-08 File upload safety (M):** max 25 MB; allow-list of file types checked on content, not only on extension; random storage names; files never served from the web root and always downloaded with Content-Disposition: attachment. A virus-scan hook is Could.
- **SEC-09 Audit log (S):** sign-in, failed sign-in, password change and membership changes are logged with user id, time and IP address and kept for 90 days.
- **SEC-10 Privacy, GDPR (S):** a user can export their data and delete their account. After deletion, authored messages remain and are shown as "Deleted user". Personal data collected: email, display name, optional profile picture, and IP addresses in the audit log only.

4.6 Usability and accessibility

The platform must provide a consistent, understandable and accessible user experience across its main workflows.

- **USE-01 Consistency (M):** screens, navigation and labels follow the wireframes in Section 6.
- **USE-02 Feedback (M):** every action shows a visible reaction within 100 ms; sending a message uses optimistic UI with the states Sending, Sent and Failed (with a Retry action).
- **USE-03 Error messages (M):** human-readable, shown next to the form field they belong to, with no technical details.
- **USE-04 Accessibility (S):** WCAG 2.1 AA contrast, keyboard access to the main flows, text labels for icon-only buttons (image, attach, workflow, send).
- **USE-05 Connection state (S):** when the WebSocket is disconnected, a "Reconnecting…" banner is shown within 2 s and disappears after REL-05 recovery.
- **USE-06 Time handling (M):** timestamps are stored in UTC and shown in the user's local time zone.

4.7 Compatibility and portability

The platform must operate in supported web browsers and use deployment and communication standards that simplify installation and integration.

- **COMP-01 Browsers (M):** latest two major versions of Chrome, Firefox, Edge and Safari.
- **COMP-02 Server delivery (M):** runs on Linux as containers (application server, PostgreSQL), started with one command (docker compose up); configuration only through environment variables.
- **COMP-03 Open standards (M):** REST/JSON described in OpenAPI 3 (Section 8), JSON events over WebSocket, SMTP for email.

4.8 Maintainability, testability and observability

The system must be structured so that developers can maintain, test, monitor and extend it without unnecessary coupling between modules.

- **MAIN-01 Modularity (M):** code follows the structure in Section 5: bounded contexts that talk to each other only through application interfaces and domain events.
- **MAIN-02 Tests (S):** ≥ 70 % line coverage of domain and application layers; an integration test for each REST endpoint and WebSocket event; one end-to-end test of the send-message flow; one load test for PERF-01 and PERF-08.
- **MAIN-03 Schema migrations (M):** versioned database migrations (e.g. Flyway or Liquibase); no manual schema changes.
- **MAIN-04 Logging (M):** structured JSON logs with a correlation id per request; passwords, tokens and message bodies are never logged.
- **MAIN-05 Health and metrics (S):** a /health endpoint; counters for connected users, messages per minute, error rate and used file storage.
- **MAIN-06 Continuous integration (S):** build and tests run automatically on every commit.

4.9 System limits and data retention

Content limits

- Message text: 1 to 4,000 characters.
- Attachments per message: up to 10.

- File size: 25 MB.
- Channel name: 3 to 50 characters, unique (case-insensitive).
- Channel description: up to 500 characters.
- Display name: 2 to 50 characters.
- Workflow title: 1 to 100 characters; workflow description: 1 to 2,000 characters.

Per-user limits. These keep one user from using up a shared resource. Exceeding one returns a readable error (USE-03).

- File storage: 500 MB in total, the profile picture included, counted as the sum of size_bytes of the user's uploaded files. 1,000 users × 500 MB = 500 GB, the size of the file volume.
- Profile picture: 2 MB, image types only, because it is shown in many places and must load fast.
- Messages: 30 per minute (SEC-07), to protect the burst capacity in 4.1.2.
- Failed sign-ins: 5 per 15 minutes (SEC-07), as brute-force protection.
- Code resend: once per 60 s (SEC-04), to limit email traffic.
- Open connections: 5 WebSocket connections (browser tabs), to bound the connection registry.

Data retention

- Messages and files: kept until deleted by the author or an administrator; there is no automatic expiry. A deleted file frees the user's quota.
- Audit log: kept for 90 days (SEC-09).
- One-time codes: stored hashed and deleted after use or after 10 minutes.
- Refresh tokens: deleted 7 days after issue, or on sign-out or password change.
- Notifications: kept for 90 days.
- Backups: kept for 7 days (REL-07).
- Deleted account: email and display name are removed; authored messages are kept and shown as "Deleted user" (SEC-10).

5. High level architecture

5.1 Architectural overview

The platform is a client-server system with a real-time push channel. The main architectural decisions are:

1. **Client-server over HTTPS.** A browser single-page application (SPA) talks to one application server, which is the single source of truth.
2. **Two communication channels.** REST/JSON carries commands and queries (request-response). One WebSocket per client carries server-to-client events and heartbeats.
3. **Modular monolith organised by domain-driven design.** The domain is split into bounded contexts (5.3). Each context is a separate package with its own domain model**.** This is simple enough to implement in one semester, and the contexts can be split into services later.
4. **Event-driven inside the server.** Contexts publish domain events to an EventBus; other contexts react through subscribers (Observer pattern). Notifications, real-time fan-out and search indexing are subscribers, not calls from the message code.
5. **Ports and adapters.** Domain and application code depend only on interfaces (repositories, email sender, file storage, event bus). The infrastructure package implements them.
6. **Explicit lifecycles with enums.** User presence and workflow status are small state machines (5.5.1, 5.5.2). Each is one enum plus a guarded transition method, not a set of state classes.

5.2 System structure and deployment

The system consists of a browser client, an application server, a relational database, a file volume and an SMTP email service (Figure 5.1). Audio and video calls are out of scope (1.4), so there is no media server and no WebRTC component.

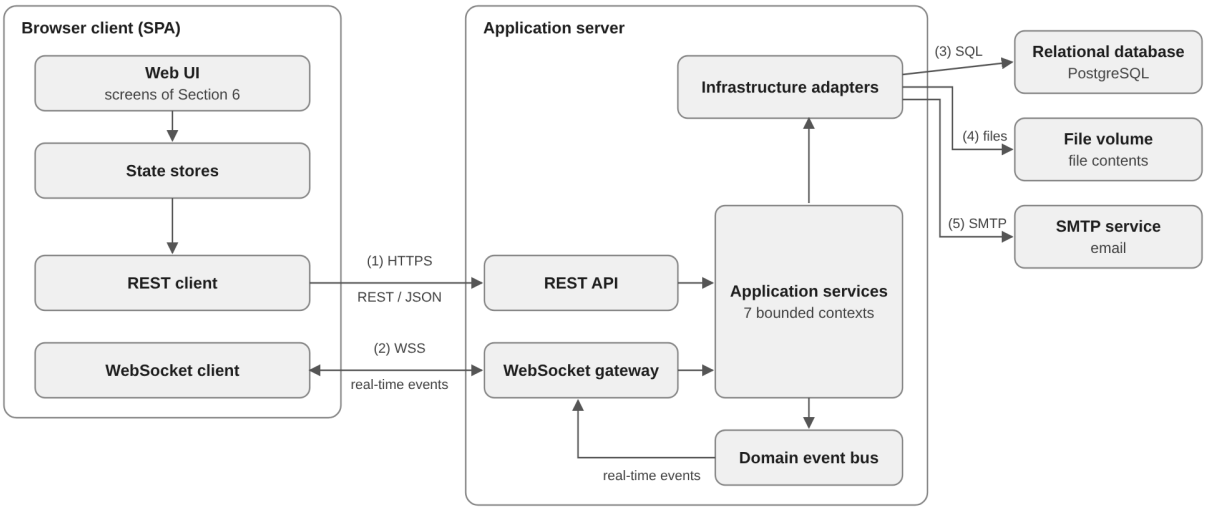


Figure 5.1. System architecture: components and the protocol on each connection.

The browser client contains the user interface, state stores, a REST client and a WebSocket client. The application server contains the REST API, the WebSocket gateway, the application services, the domain event bus and the infrastructure adapters. The relational database (PostgreSQL) stores all persistent application data; uploaded file contents are stored on a separate file volume (4.1.3).

Figure explanation. The browser uses two channels to the server. REST requests (1) handle commands and queries such as signing in, sending messages and managing workflows. The WebSocket (2) delivers real-time events: new messages, notifications and presence changes. Inside the server every request goes through an application service, which uses the database (3), the file volume (4) and the SMTP service (5) only through ports.

5.3 Bounded contexts

The server is divided into seven bounded contexts. Each context has one clear responsibility and owns its own data. Contexts communicate in two ways, shown in two separate figures so that each stays readable:

- **Synchronous calls** through a small query interface (Figure 5.2): used when a context needs an answer now, for example "is this user a member of this conversation?".
- **Domain events** through the EventBus (Figure 5.3): used when a context announces that

something happened and does not care who reacts.

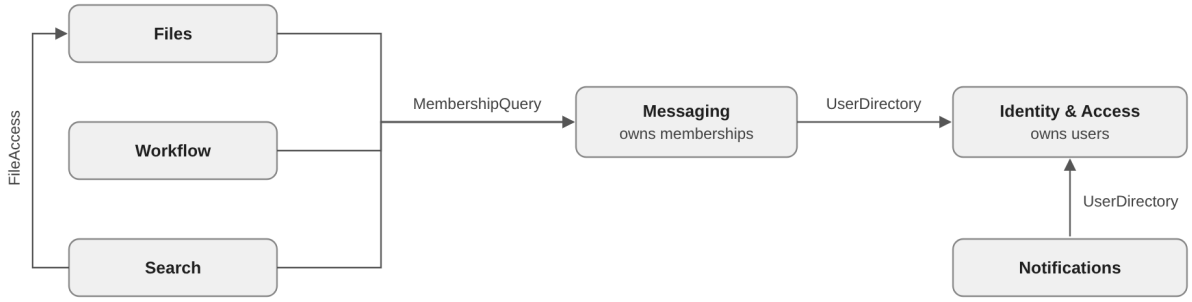


Figure 5.2. Synchronous dependencies between contexts. An arrow means "calls the interface of".

Figure 5.2 shows that Messaging is the centre of the model: Files, Workflow and Search all ask Messaging about membership through MembershipQuery, and nobody reads Messaging's tables directly (MAIN-01). Presence is not shown because it calls no other context; it only reacts to events (Figure 5.3).

		Consumers			
Publisher	**Event**	**Notifications**	**Search**	**Presence**	**RealtimeBroadcaster**
Messaging	MessageSent	●	●		●
	MemberJoined	●			
Files	FileUploaded		●		
Workflow	WorkflowAssigned	●			
	WorkflowStatusChanged	●			●
	WorkflowRejected	●			●
WebSocket gateway	ConnectionOpened / Closed			●	
Presence	PresenceChanged				●
Notifications	NotificationCreated				●

Figure 5.3. Domain events. Each row is an event with its publisher; a dot marks a component that consumes it.

Figure 5.3 shows that Notifications and the real-time broadcaster only listen: they never call the publishing context back.

Context responsibilities

- **Identity & Access:** owns User, OneTimeCode, RefreshToken and AuditEntry. Publishes *UserRegistered, UserDeleted, PasswordChanged.* Offers *UserDirectory* (get/exists).
- **Messaging:** owns Conversation (channel or DM), Membership and Message. Publishes *MessageSent, MemberJoined, MemberLeft*. Offers *MembershipQuery.isMember(userId, conversationId)*.
- **Presence:** owns the presence status of each user. Publishes *PresenceChanged*. Consumes *ConnectionOpened*, *ConnectionClosed*. Offers *PresenceQuery*.
- **Files:** owns file metadata (content is in file storage). Publishes *FileUploaded*. Offers *FileAccess*, which checks membership through Messaging.
- **Workflow:** owns Workflow (a task attached to a channel). Publishes *WorkflowAssigned*, *WorkflowStatusChanged, WorkflowRejected*. Uses *MembershipQuery*.
- **Notifications:** owns Notification. Publishes *NotificationCreated*. Consumes *MessageSent*, *MemberJoined*, *WorkflowAssigned*, *WorkflowStatusChanged, WorkflowRejected*. Offers *NotificationQuery*.
- **Search:** owns the search index of messages and file names. Consumes *MessageSent*, *FileUploaded*. Offers *SearchService*, which filters results by membership.

ConnectionOpened and ConnectionClosed are published by the WebSocket gateway in the api package. Real-time delivery is not a context: RealtimeBroadcaster (infrastructure) subscribes to every event that must reach clients and pushes it to the recipients' WebSocket connections.

5.4 Package Structure

The codebase is divided into server and client packages. This keeps the user interface, the business rules and the infrastructure from becoming tightly coupled.

5.4.1. Server Packages

The server has four top-level packages under teamchat (Figure 5.4).

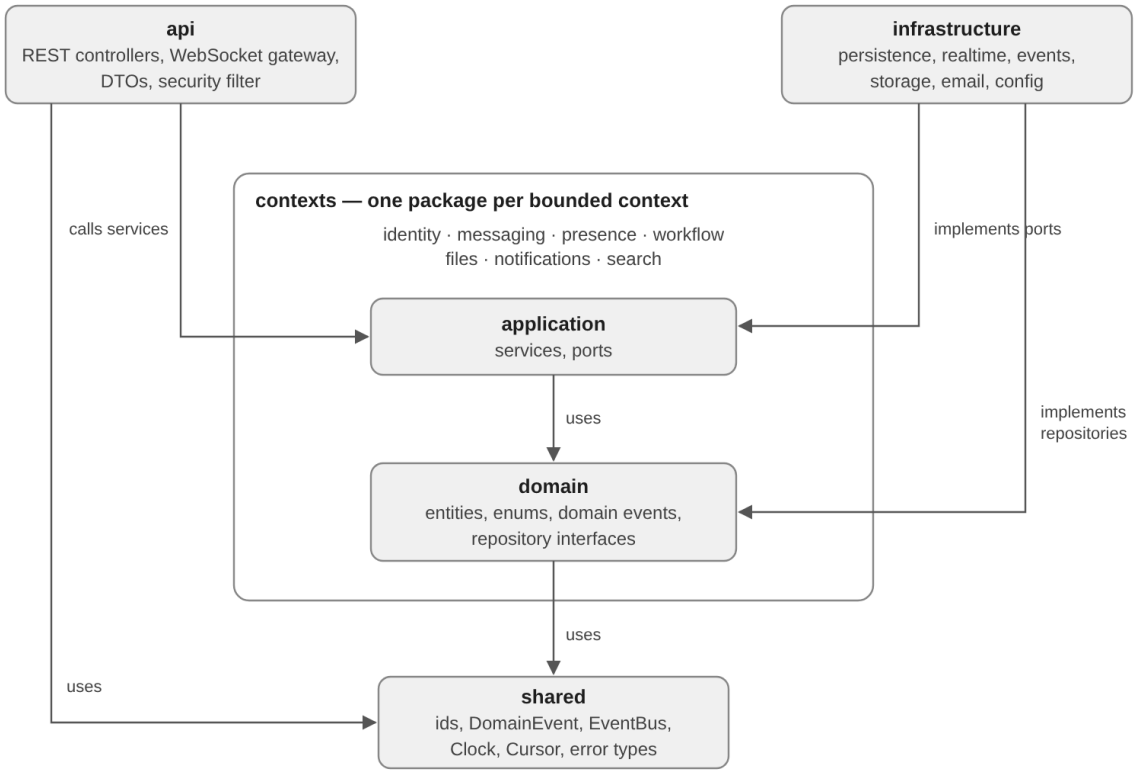


Figure 5.4. Server packages and the allowed dependencies.

Every context package has exactly two layers: domain (entities, enums, domain events, repository interfaces) and application (services and ports such as EmailSender and FileStorage). Controllers and adapters live outside the contexts, in api and infrastructure. The dependency rules are:

- api depends only on application. It never touches domain entities directly; it returns DTOs.
- domain depends on nothing except shared.
- infrastructure implements the ports and depends on domain and application.
- Contexts never access each other's tables or domain classes; they use only the interfaces and events listed in 5.3 (MAIN-01).

5.4.2 Client packages

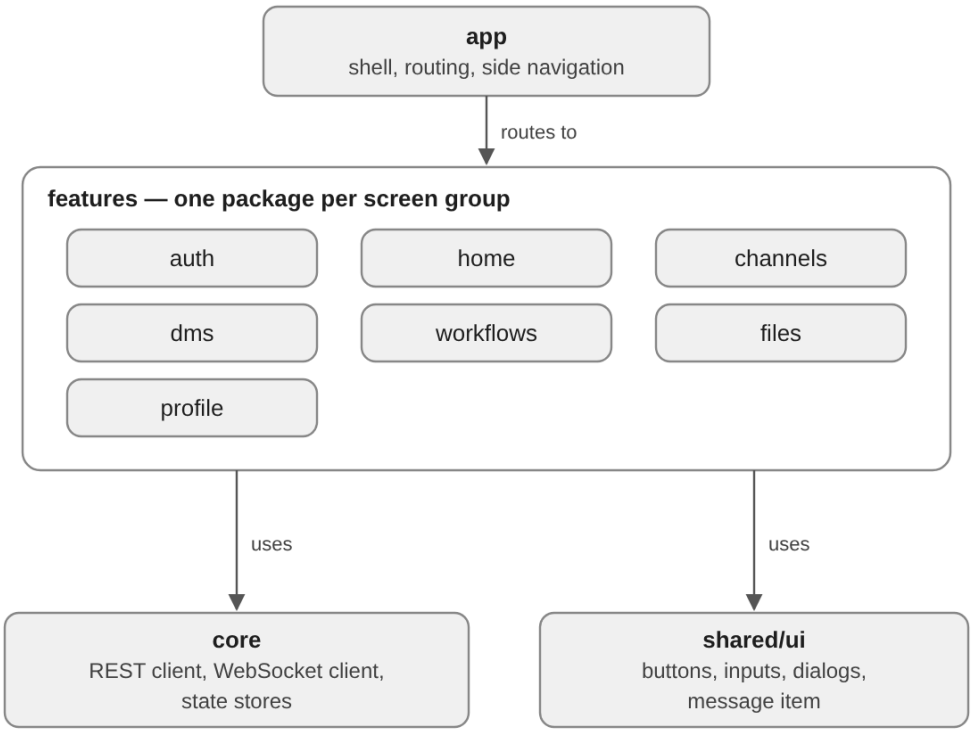


Figure 5.5. Client packages.

- **app** holds the application shell, routing and the side navigation; it routes to the feature packages.
- **features** has one package per screen group: auth, home (main page, notifications, help), channels, dms, workflows, files and profile. Search is not a separate screen: each list page has its own Search field (Section 6).
- **core** holds code shared by all features: the REST client, the WebSocket client with reconnection (REL-05) and the state stores.
- **shared/ui** holds reusable components: buttons, inputs, dialogs and the message item.

Features may use core and shared/ui, but never another feature. core never depends on features.

5.5 Design patterns and key mechanisms

Design patterns

- **Observer:** used in the *EventBus* with its subscribers (Notifications, RealtimeBroadcaster,

SearchIndexHandler). Publishers do not know their consumers, so new reactions can be added without changing message code.

- **Ports and adapters:** used for repositories, EmailSender, FileStorage and EventBus. Infrastructure can be replaced without touching domain code (SCAL-02).
- **Repository:** one per aggregate. It hides persistence from the domain.
- **Facade:** application services such as *MessageService* give one entry point per use case for both REST and WebSocket.

The State pattern is deliberately not used. Presence and workflow have few states and no behaviour that differs per state apart from deciding which transition is allowed, so an enum with a transition check is simpler, shorter and easier to test.

5.5.1 Presence state machine

Presence is a simple enum PresenceStatus with three values: ONLINE, AWAY and OFFLINE (FR-32). It is kept only in server memory (Section 7). PresenceService reacts to events and computes the next status from the rules below; Figure 5.6 shows the same rules.

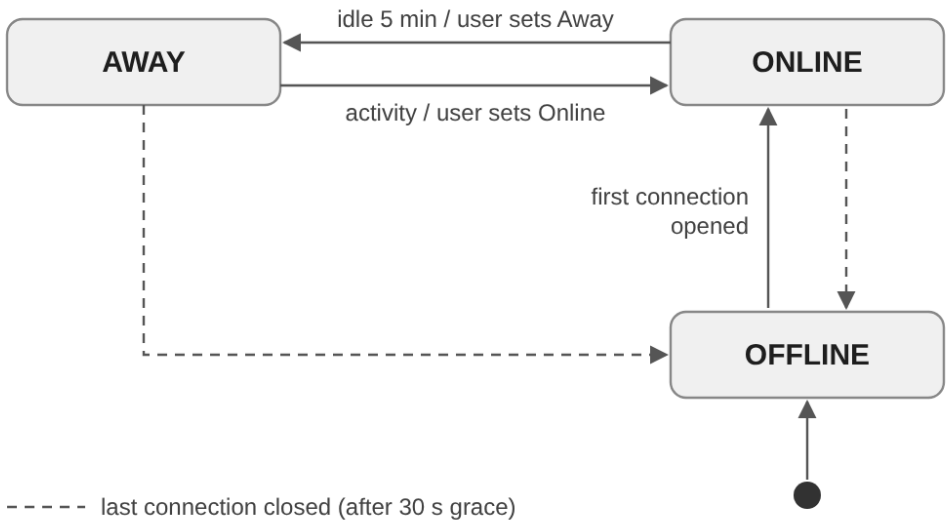


Figure 5.6. Presence state machine

- ***OFFLINE → ONLINE:*** *the first connection is opened (ConnectionOpened).*

- ***ONLINE → AWAY:*** *no activity for 5 min, or the user sets Away (FR-33).*
- ***AWAY → ONLINE:*** *user activity, or the user sets Online.*
- ***ONLINE or AWAY → OFFLINE:*** *the last connection is closed and not reopened within 30 s.*

A status set manually to Away stays Away until the user changes it; activity does not override it. Every change publishes PresenceChanged. The 30 s grace period prevents flicker when a page is refreshed.

The three statuses have no behaviour of their own; only the next status differs, and the rules above defines it in one place.

5.5.2 Workflow state machine

Workflow status is the enum WorkflowStatus with three values, exactly as in the data model (Section 7.3): OPEN, IN_PROGRESS and COMPLETED. In the UI and in Sections 2–3 these appear as To Do, In Progress and Done. Figure 5.7 shows the only allowed transitions.



Figure 5.7. Workflow state machine.

- **OPEN → IN_PROGRESS:** the assignee presses "Start" on the Workflow page (called "On it!" in Sections 2 and 7). If the workflow has no assignee yet, the channel member who presses it becomes the assignee (claim, FR-35).
- **IN_PROGRESS → COMPLETED:** the assignee presses "Complete"; completedAt is set and the workflow moves to the Completed list.
- **Reject:** while the workflow is OPEN, the assignee can press "Reject". The assignee is cleared, the status stays OPEN, the workflow leaves the assignee's Workflows page and the assigner is notified. Reject changes the assignee, not the status, so the state machine keeps three states.

- No other transitions exist (no cancel, no reopen). Only the assignee may press these buttons; anyone else gets an error.

The rules live in three methods of the Workflow entity, start(actorId), complete(actorId) and reject(actorId), each checking the current status and the actor. Start and Complete publish WorkflowStatusChanged; Reject publishes WorkflowRejected. Importance (ASAP, SOON, EVENTUALLY) is a plain attribute and does not affect the status.

5.6 Class diagrams

The class diagrams describe the main classes that must be implemented, with key attributes, operations, relationships and interfaces. Constructors and ordinary getters and setters are omitted.

The diagrams use long for identifiers, matching the server-generated integer IDs of the data model (Section 7) and the API (Section 8.1). They use String for text, Instant for timestamps (UTC, USE-06), int for counters and boolean for flags. Attribute names are the camelCase form of the column names in Section 7 (conversation_id → conversationId).

5.6.1 Identity & Access and Messaging

Identity & Access manages user accounts, one-time codes, refresh tokens and the audit log. Messaging manages conversations (channels and DMs), memberships, messages and attachments (Figure 5.8).

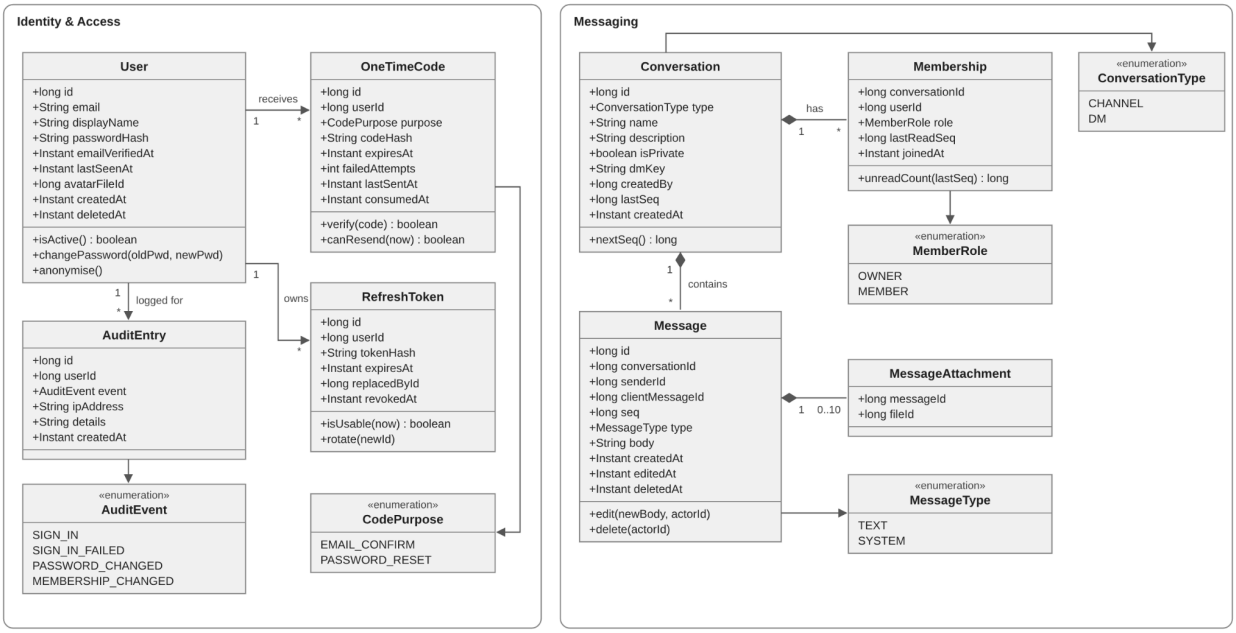


Figure 5.8. Identity & Access and Messaging classes.

A channel and a DM are one class, Conversation, distinguished by type, as in Section 7.2. Conversation.nextSeq() increments lastSeq inside the send transaction and gives the new message its seq (REL-04).

5.6.2 Application Services and Ports

Each bounded context provides application services that implement its use cases (Figure 5.9). Services are the entry points for API requests and coordinate domain logic, data access and communication with other contexts. A use case that modifies persistent data runs in one database transaction.

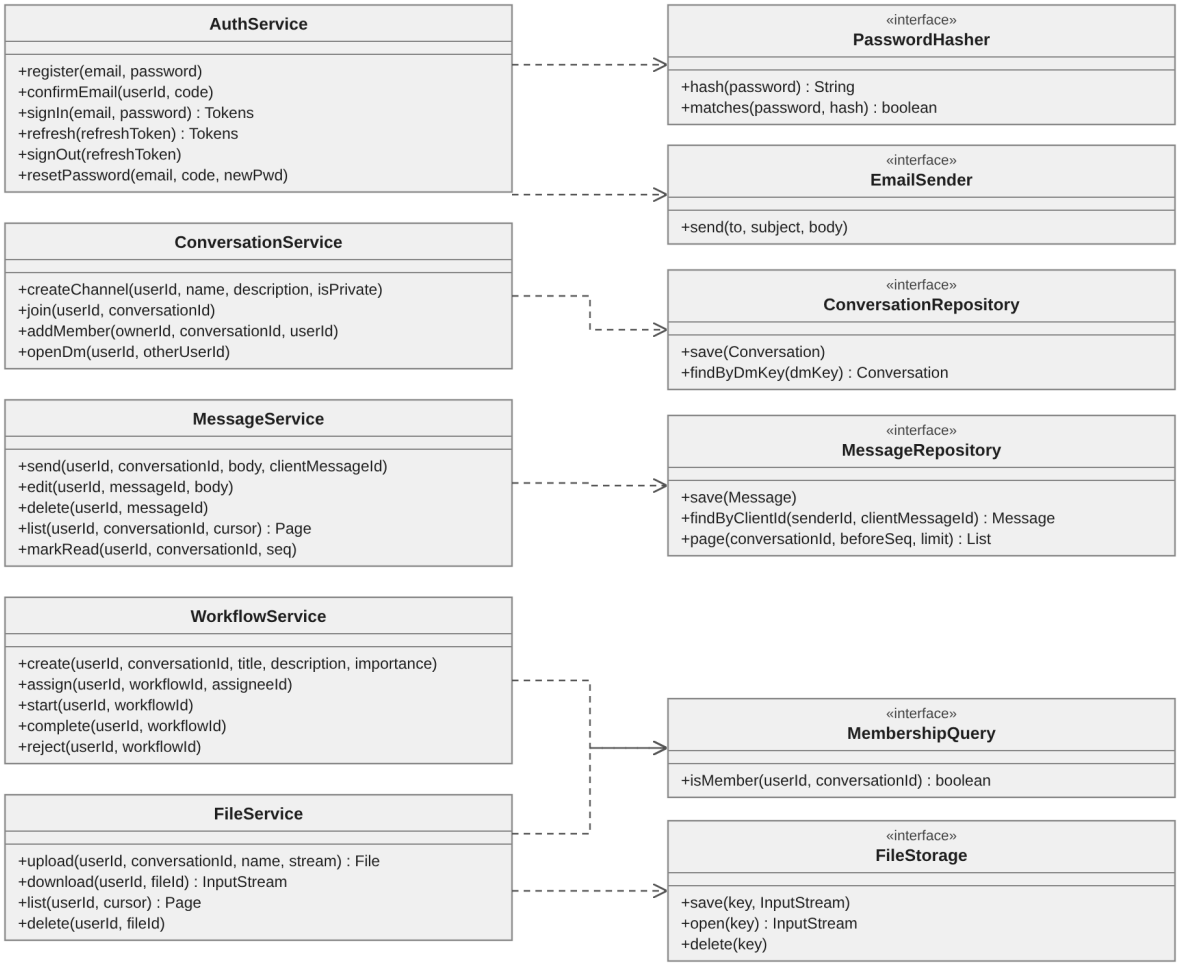


Figure 5.9. Application services and the ports they use.

The main application services are:

- **AuthService:** registration, email confirmation, sign-in, token refresh, sign-out and password reset.
- **UserProfileService:** reads, updates and searches user profiles.
- **ConversationService:** creates channels, manages memberships and opens DMs.
- **MessageService:** sends, edits, deletes and lists messages, and marks conversations as read.
- **PresenceService:** tracks connections and computes the presence status (5.5.1).
- **WorkflowService:** creates and assigns workflows and performs Start, Complete and Reject (5.5.2).

- **FileService:** uploads (streamed, PERF-07), downloads, lists and deletes files and checks the user's quota (4.9).
- **NotificationService:** lists notifications and marks them as read.
- **SearchService:** searches messages and file names (FR-40). The Search fields on the Channels, DMS, Workflows and Files pages (Section 6) filter those lists by name through their own services.

Ports are interfaces for repositories and external functionality: password hashing, token creation, email delivery, file storage, event publishing and real-time broadcasting. Cross-context access is explicit: Workflow and Files use MembershipQuery from Messaging and never its repositories or domain classes.

To keep Figure 5.9 readable, the EventBus port is not drawn there; every service that changes data publishes its events through it (Figure 5.11).

5.6.3 Presence and Workflow

Both lifecycles are modelled with enums (5.5.1, 5.5.2), so each context needs only one entity and one enum (Figure 5.10).

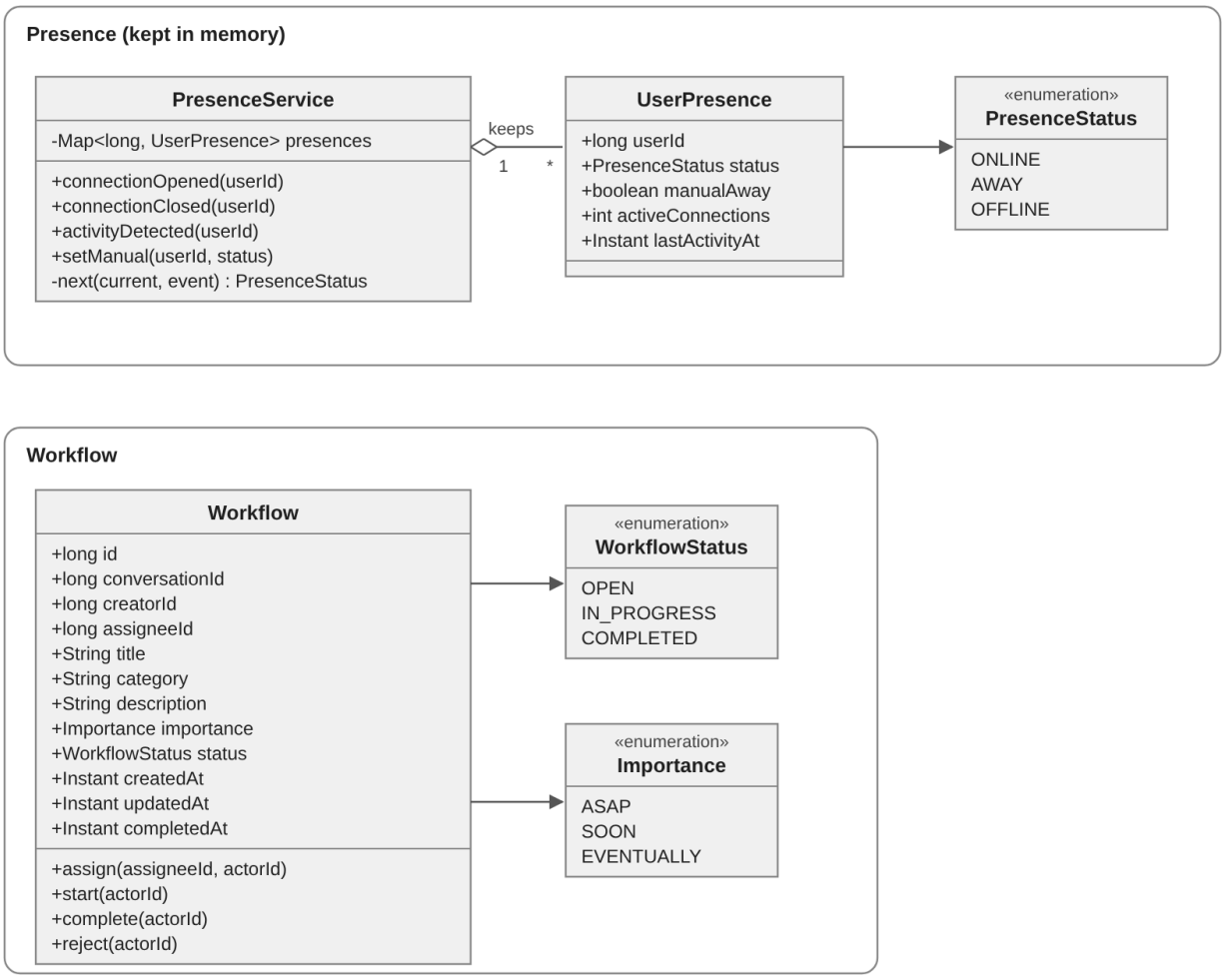


Figure 5.10. Presence and Workflow classes.

UserPresence is an in-memory record per user; PresenceService holds the transition rules . Workflow holds its own rules in start(), complete() and reject().

5.6.4 Domain Events and Notifications

The Observer pattern is implemented through EventBus and EventHandler (Figure 5.11). A context publishes a domain event without knowing which components will react to it, and each subscriber handles the event independently.

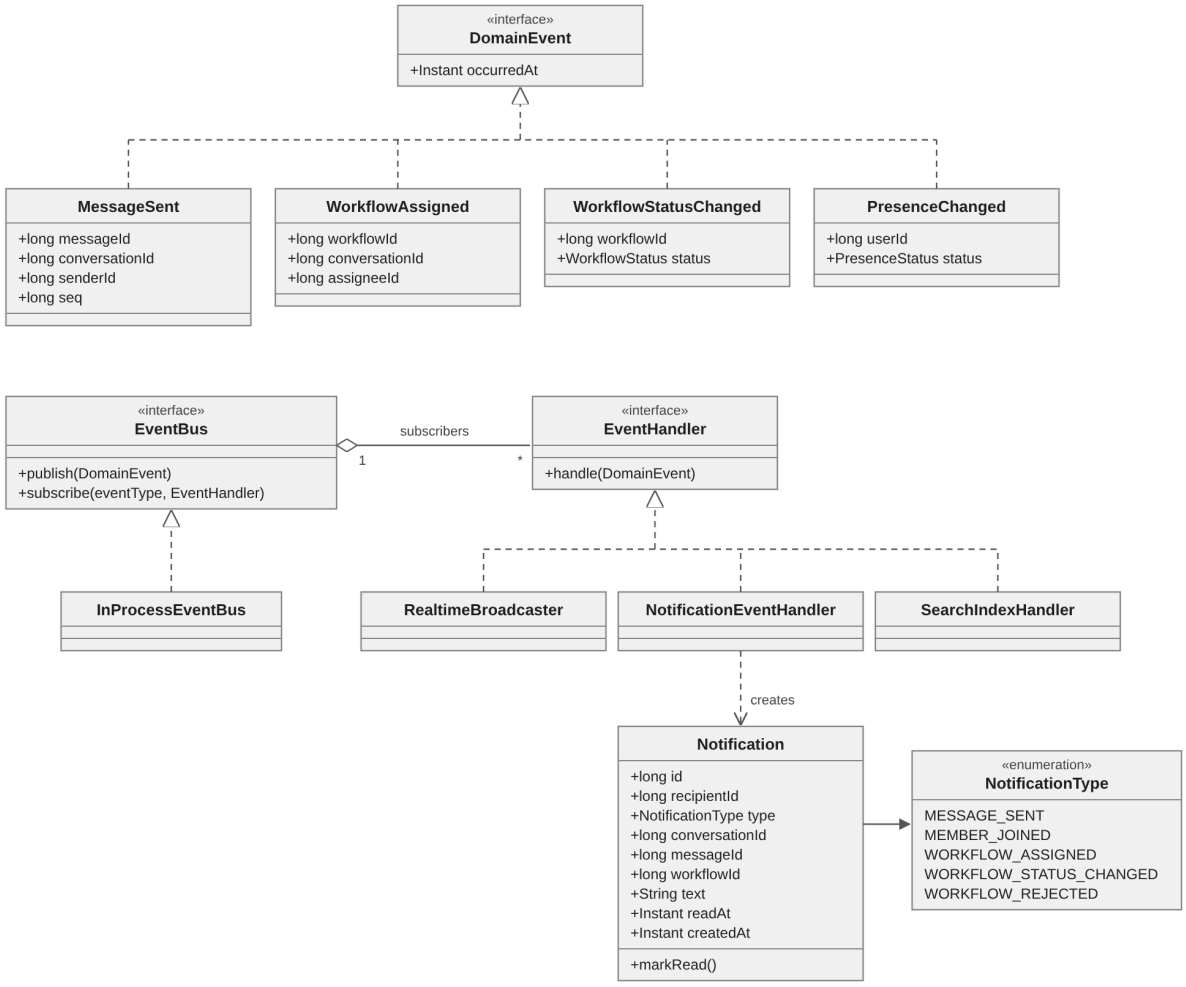


Figure 5.11. Domain events, the event bus and its subscribers.

For example, when a message is sent, Messaging publishes MessageSent. The event triggers a real-time broadcast, a notification and a search index update, without MessageService calling those components directly. NotificationEventHandler stores one Notification per recipient (fields as in Section 7.3) and publishes NotificationCreated.

5.6.5 Files

The Files context manages file metadata and access to the stored content (Figure 5.12). The content itself lives on the file volume and is reached only through the FileStorage port.

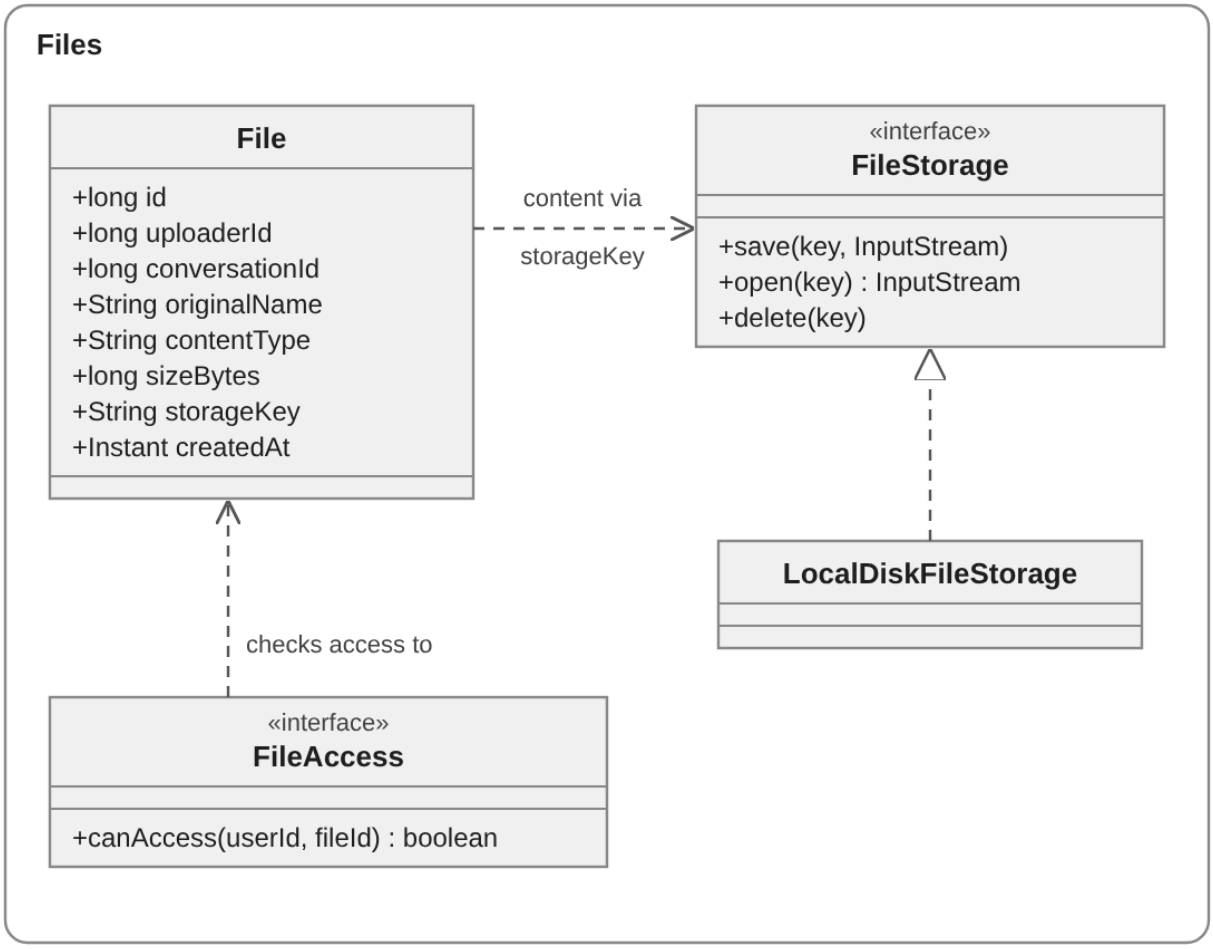


Figure 5.12. Files classes.

FileStorage works with streams, never with byte arrays, which is what makes PERF-07 possible. FileAccess lets other contexts check whether a user may open a file without reading the Files tables.

5.7 Runtime Views

Runtime views describe how the components interact during important user operations. The detailed sequence diagrams for these operations are to be included in the team's sequence-diagram contribution.

5.7.1 Sending a Message

The client sends the message body and a clientMessageId through the REST API. The API calls

MessageService, which checks membership, validates the message and checks whether the request is a retry of an existing message (REL-03).

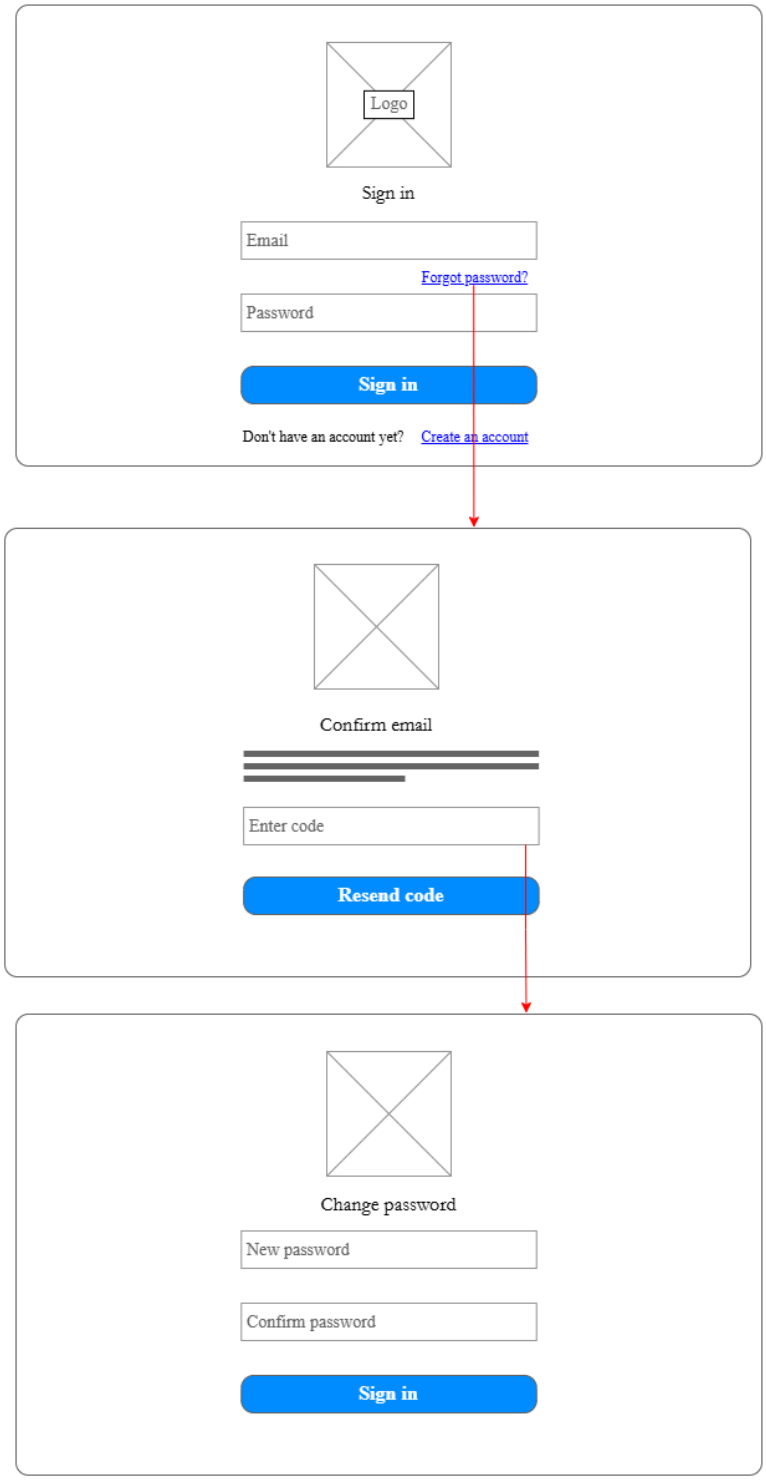
In one transaction the service takes the next sequence number from the conversation and saves the message. After the commit it publishes MessageSent. The real-time broadcaster and the notification handler process the event and update the recipients' clients through the WebSocket.

If a request fails or times out, the client keeps the message in the Failed state and retries with the same clientMessageId, so no duplicate is created.

The detailed sequence diagrams should show the participants, the message order, the membership checks, the event publication and the real-time delivery to the recipients.

6. UI/UX wireframes

1. Wireframe of the sign in window

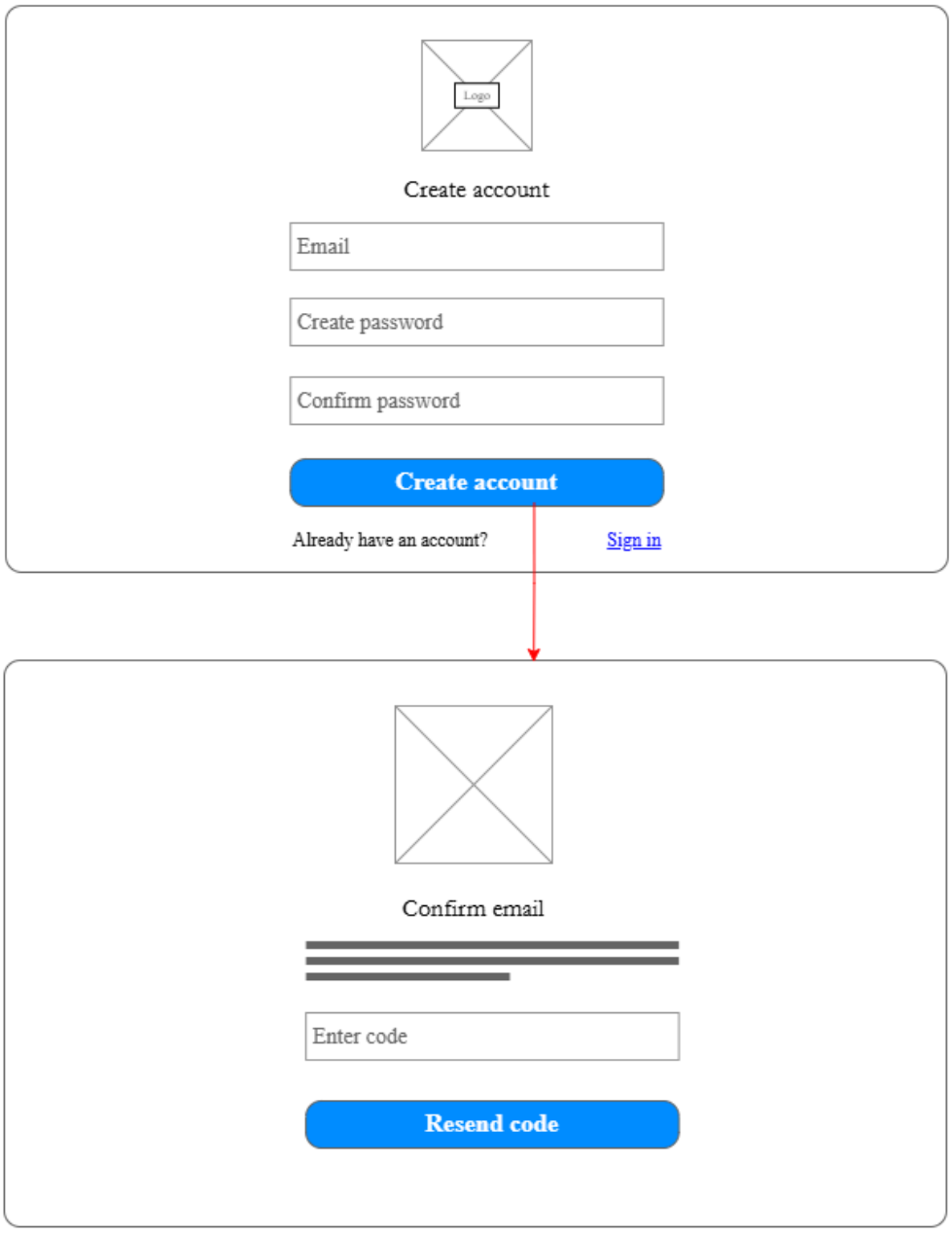


Explanation: The sign in panel provides functionality for the user to access the main functionality of the application. The “Email” and “Password” fields in the panel accept user input and provide user feedback. The “Forgot password?” link redirects the user to the “Confirm email” panel, where the user can begin password recovery process. The “Create an account” link redirects the user to the “Sign up” page. On successful sign in the “Sign in” button redirects the user to the main page.

The “Confirm email” panel allows the user to verify their identity before changing their password. The “Enter code” field accepts the verification code from the user and provides user feedback if the code is incorrect or expired. If the user did not receive the code, they can click the “Resend code” button to request a new one. When the correct code is entered, the system redirects the user to the “Change password” panel.

The “Change password” panel allows the user to set a new password for their account. The “New password” field accepts the new password and provides user feedback if it does not meet the security requirements. The “Confirm password” field accepts the repeated password and provides user feedback if the two values do not match. On successful validation, the “Sign in” button signs the user in and redirects them to the main page.

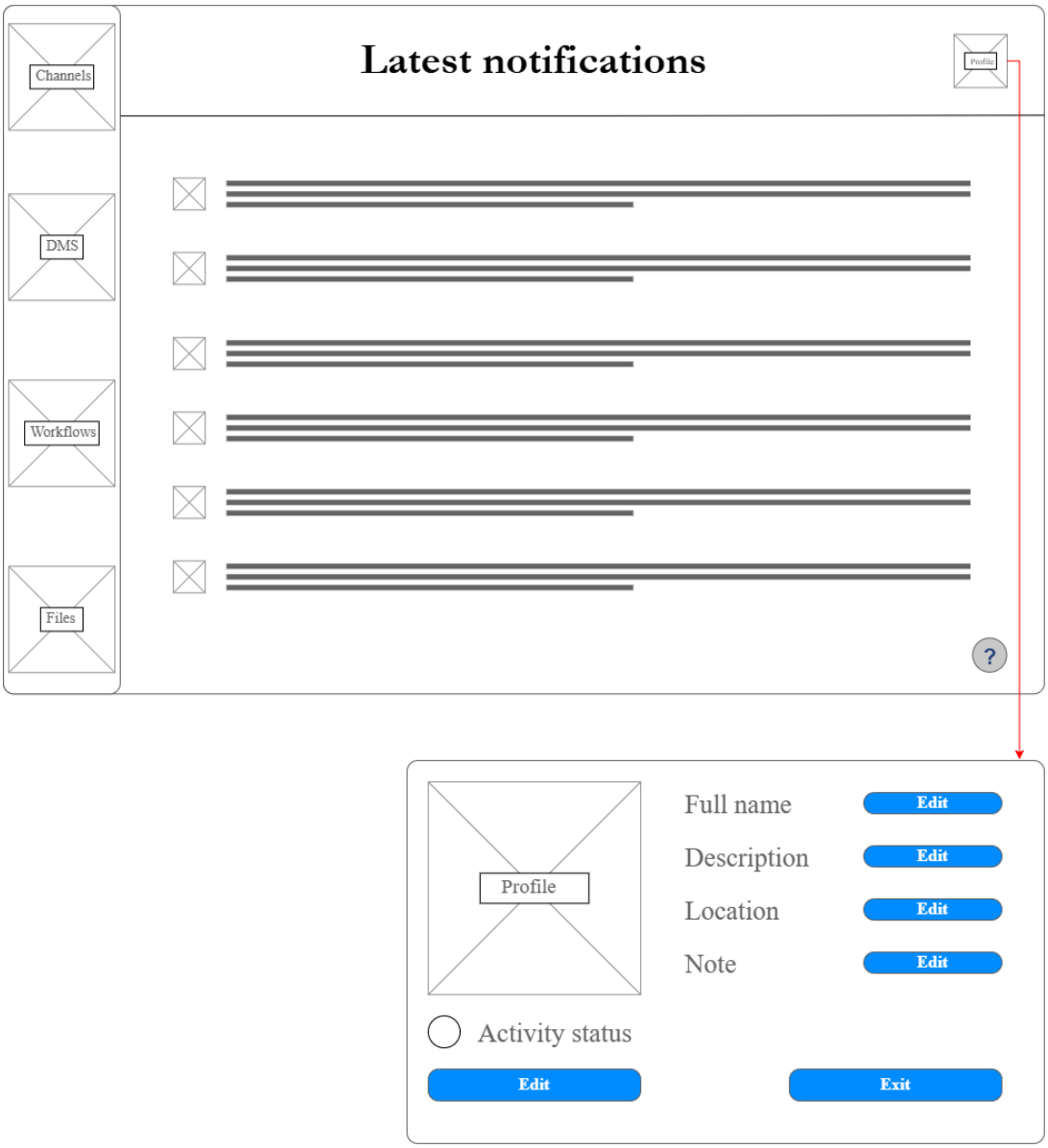
2. *Wireframe of the sign up window*



Explanation: The "Create account" panel provides functionality for the user to register in the application. The "Email", "Password" and "Confirm password" fields in the "Create account" panel accept user input and provide user feedback. The "Sign in" link redirects the user to the "Sign in" page. On successful user feedback in the "Create account" panel, the "Create

account” button redirects user to the “Confirm email” panel. In the “Confirm email” panel the “Enter code” field accepts user input and provides user feedback. On successful user feedback in the “Confirm email” panel the “Confirm” button registers the user and redirects the user to the main page. On unsuccessful user feedback in the “Confirm email” panel the “Resend code” button sends another code to the user's email address.

3. *Wireframe of the main window*



Explanation: The main page provides functionality for the user to access the main sections of the application and view the latest activity. The left side panel contains the “Channels”, “DMS”, “Workflows” and “Files” buttons. The upper side panel contains the header “Latest notifications” and the “Profile” button. The middle side panel displays all the latest notifications from all the left side panel sections.

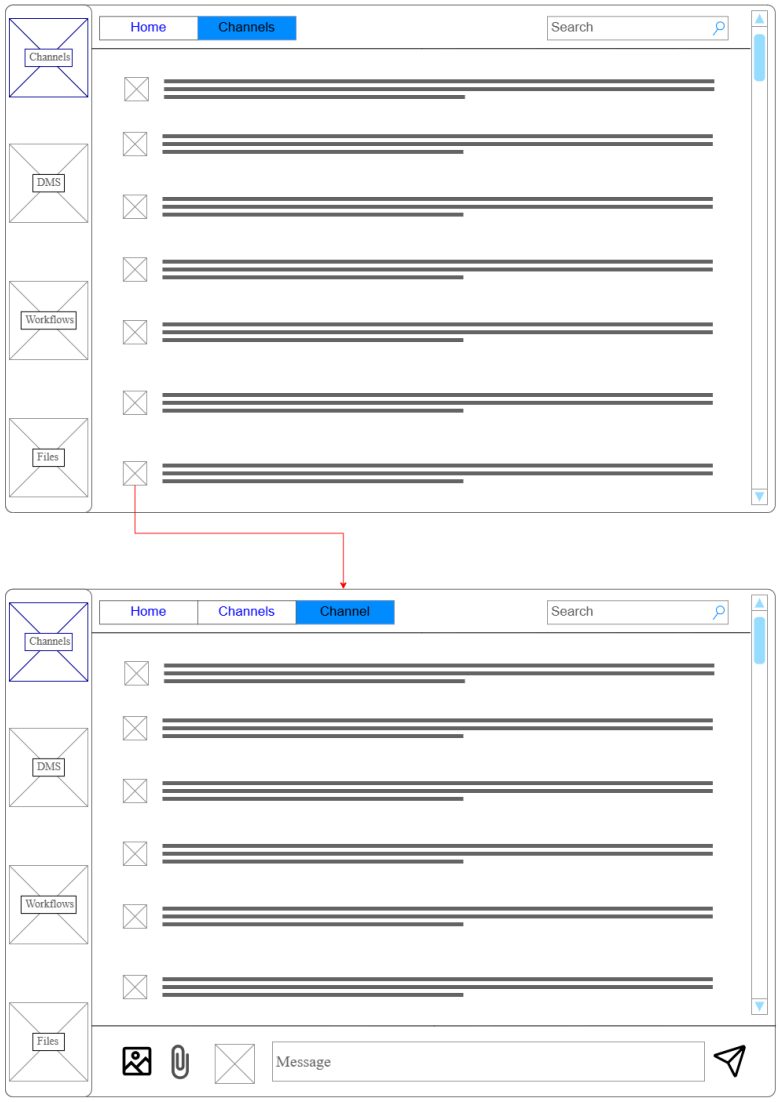
The “Channels” button redirects the user to the “Channels” page. The “DMS” button redirects the user to the “DMS” page. The “Workflows” button redirects the user to the “Workflows” page. The “Files” button redirects the user to the “Files” page. The “Profile” button redirects the user to the “Profile” page.

Each notification in the middle panel consists of an icon and a text field that provide the user with brief information about the new activity. The user can click on a notification, and the system redirects the user to the corresponding page.

The “Get help” button at the bottom right side of the middle panel opens a panel which provides application use instructions. The user can close the panel and return to the main page.

The “Profile” panel provides functionality for the user to view and manage their personal information. The “Edit” button next to the “Full name”, “Description”, “Location” and “Note” text fields provides functionality for changing the value of those fields. The “Full name” field is intended to be the user’s first and last name, the “Description” field is intended to be the user’s title (e.g. *Senior Software Engineer*, *Student*), the “Location” field is intended to be the country and / or city the user is currently in and the “Note” field is intended to be used as an additional information about the user (e.g. *I am currently on a vacation*).

4. Wireframe of viewing a channel

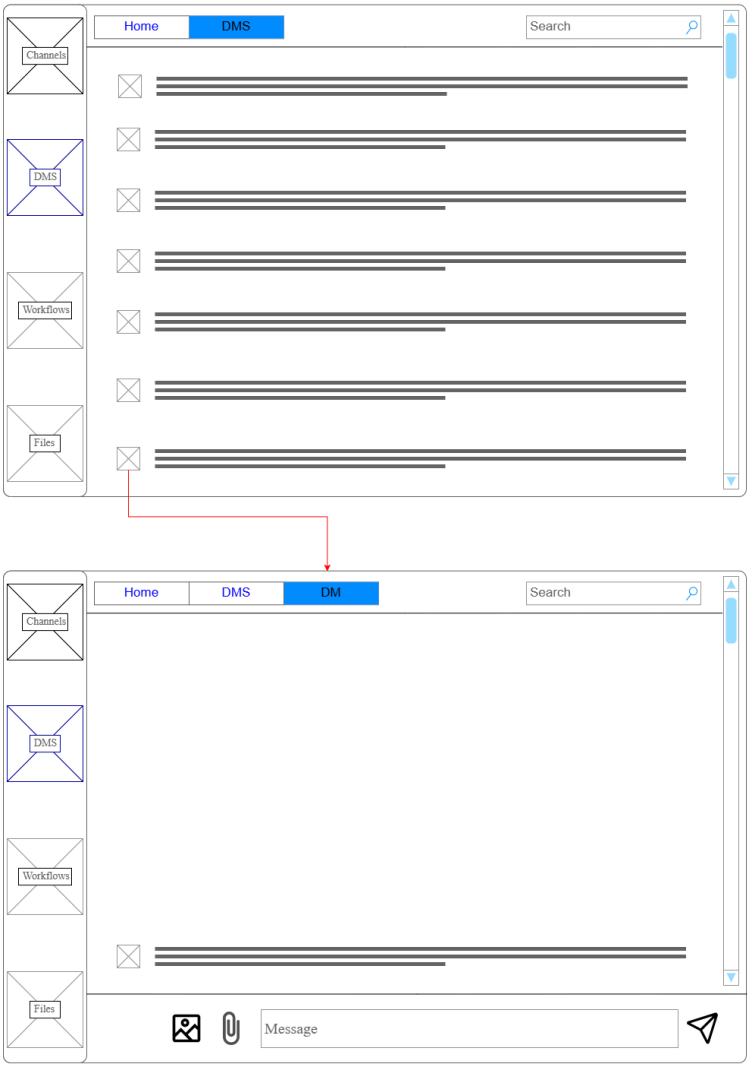


Explanation: The "Channels" page provides functionality for the user to view the list of channels available in the application and to open any of them. The upper side panel contains the breadcrumb navigation and the "Search" field, which accepts user input and searches for channels. The middle panel displays the list of channels.

The "Channel" page allows the user to view the messages of the selected channel and to communicate with other channel members. The left side panel and the "Search" field work the same way as on the "Channels" page. The "Search" field accepts user input and searches for

messages in the channel. The middle panel displays the messages of the channel, where each message consists of a group member's profile picture and a text field with the message. The bottom panel lets the user write and send messages. The “Message” field accepts user input. The “Image” button opens a panel where the user can select an image to add to the message. The “Attach” button opens a panel where the user can select a file to attach to the message. The button with the icon next to the “Attach” button provides functionality for starting a workflow. The “Send” button at the bottom right side of the page sends the message to the channel.

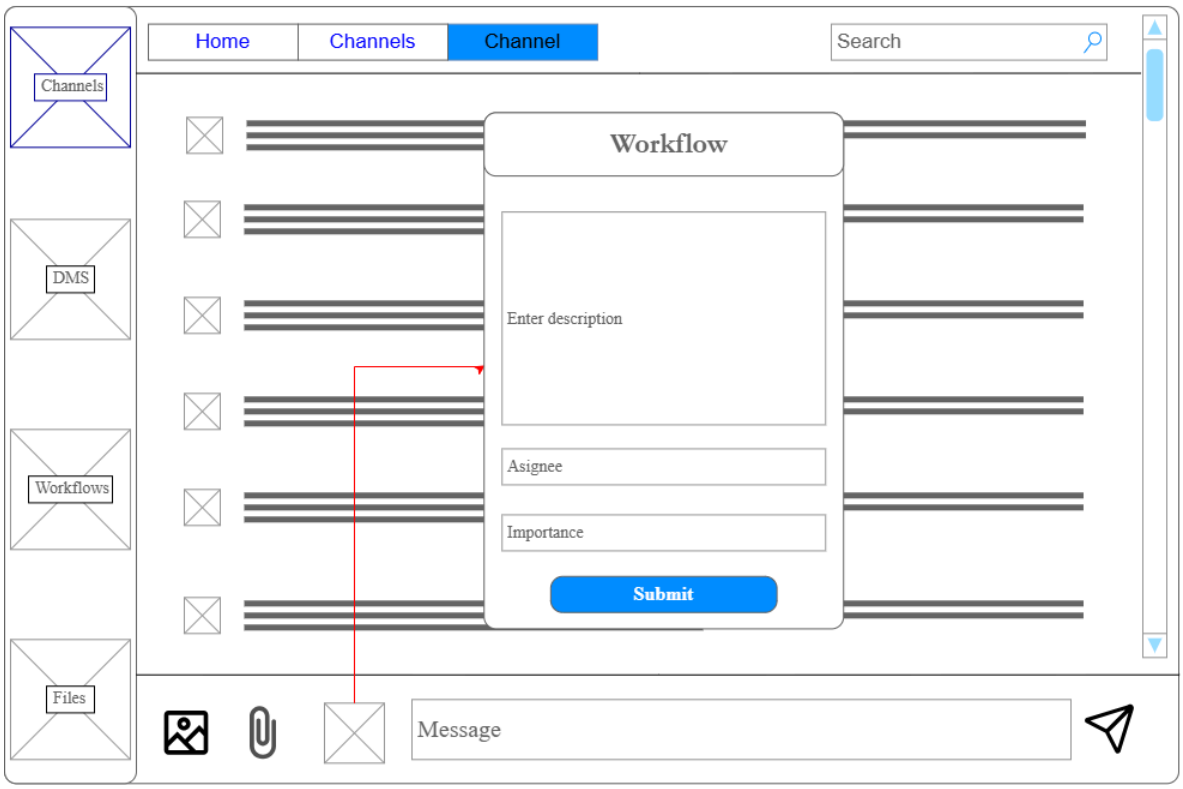
5. Wireframe of viewing a DM



Explanation: The “DMS” page provides functionality for the user to view the list of direct messages received on the account and to open them. The upper side panel contains the breadcrumb navigation and the “Search” field, which accepts user input and searches for DM’s. The middle panel displays the list of DM’s.

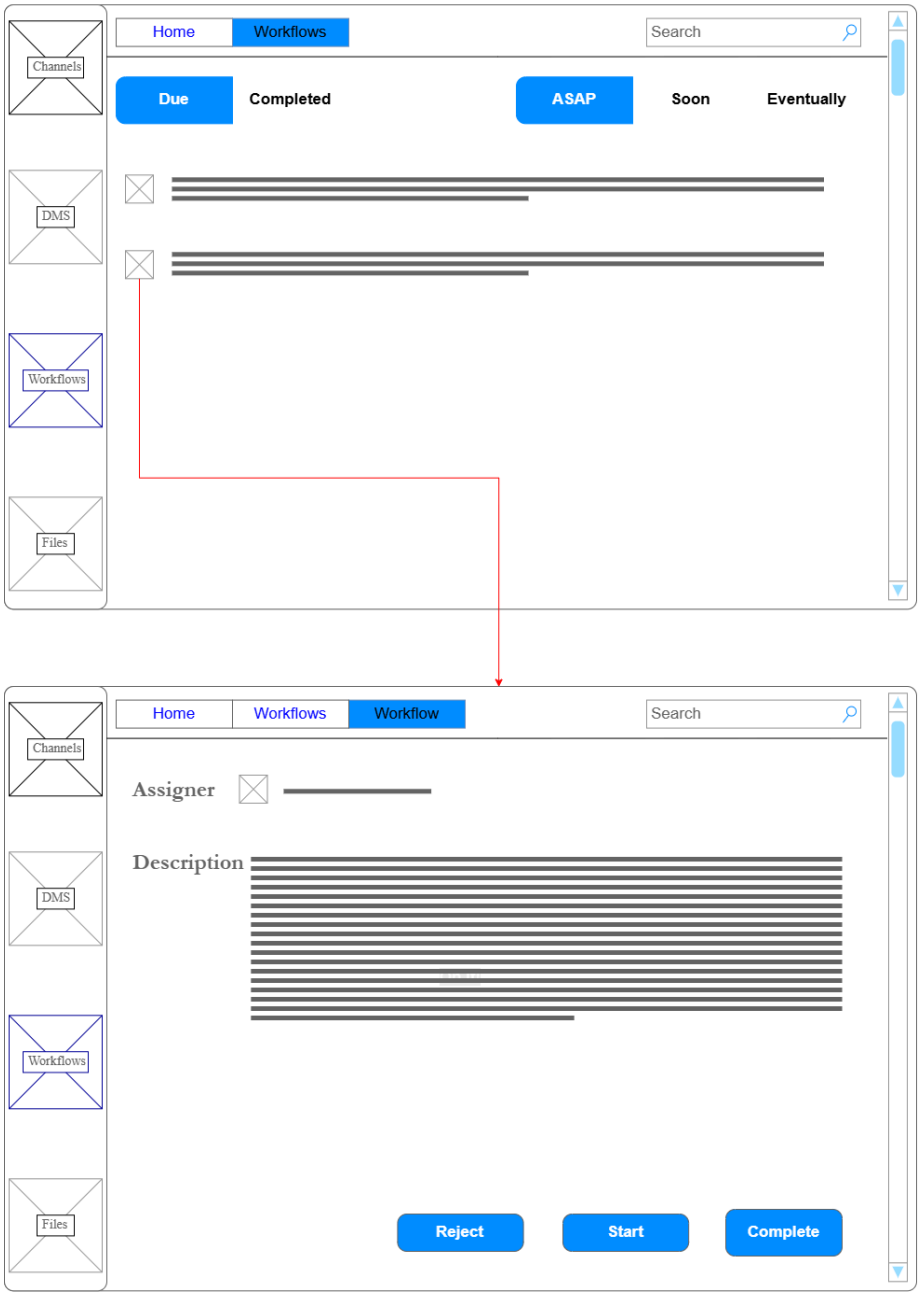
The “DM” page allows the user to view the messages of the selected conversation and to communicate with the other user. The left side panel and the “Search” field work the same way as on the “DMS” page. The middle panel displays the messages of the conversation, where each message consists of a user's profile picture and a text field with the message. The bottom panel lets the user write and send messages. The “Message” field accepts user input. The “Image” button opens a panel where the user can select an image to add to the message. The “Attach” button opens a panel where the user can select a file to attach to the message. The “Send” button at the bottom right side of the page sends the message to the channel.

6. Wireframe of creating a workflow



Explanation: The “Workflow” panel provides functionality for the user to create a new workflow directly from the “Channel” page. The “Enter description” field accepts user input and describes the task of the workflow. The “Assignee” field is a searchable selection control. When the user clicks the field, a dropdown list opens showing the members of the current channel (avatar, display name). The user can type to filter the list by name and select exactly one member. The selected member is displayed in the field and the dropdown closes. If no member is selected, the field shows the placeholder “Assignee.” The “Importance” field is a single-select dropdown. When the user clicks the field, a list of predefined priority levels opens: “ASAP”, “Soon”, “Eventually”. The fields provide user feedback if the required information is missing or entered incorrectly. On successful validation the “Submit” button creates the workflow and closes the panel.

7. Wireframe of viewing assigned workflows

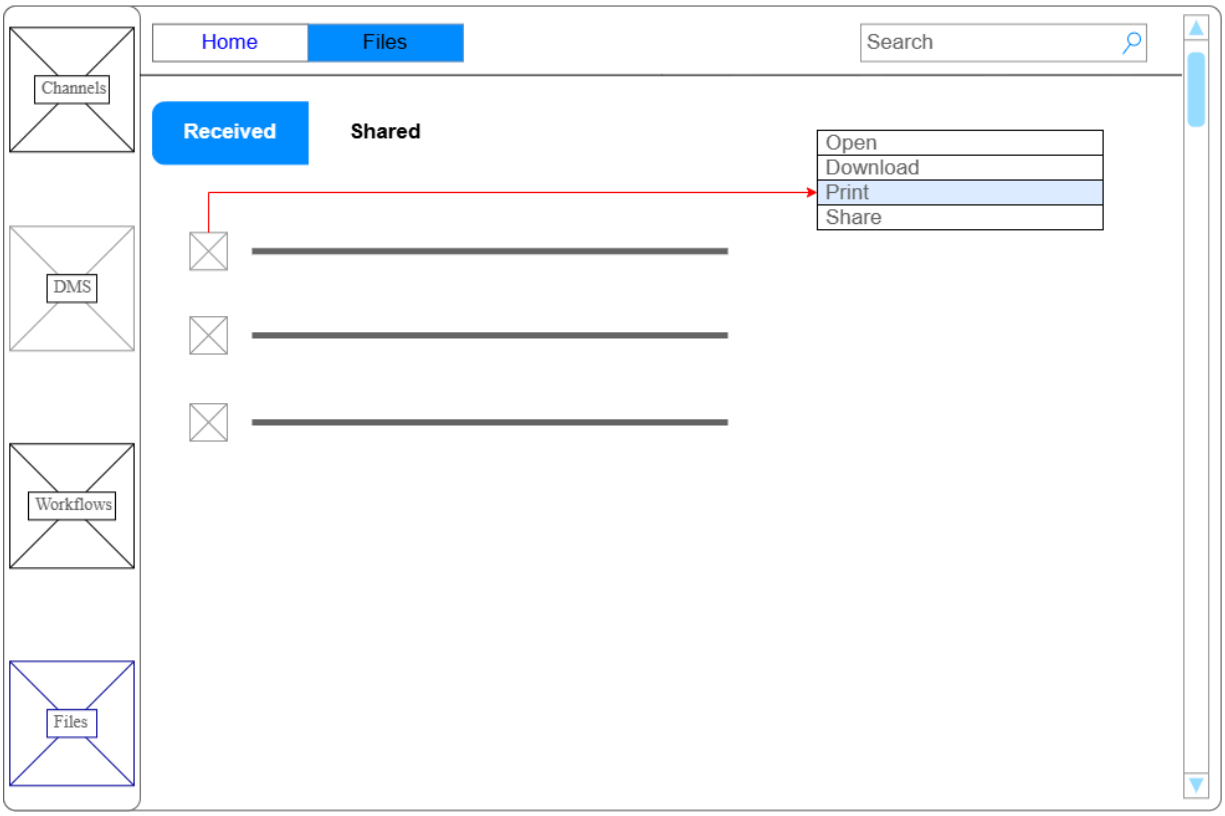


Explanation: The "Workflows" page provides functionality for the user to view the list of workflows assigned to them and to open any of them. The upper side panel contains the breadcrumb navigation and the "Search" field, which accepts user input and searches for workflows. The middle panel displays the list of workflows.

The “Due” and “Completed” buttons filter the workflows by their status. The “ASAP”, “Soon” and “Eventually” buttons filter the workflows by their importance.

The buttons in the middle panel let the user click on a workflow and redirect the user to the “Workflow” page. The “Assigner” field shows the name and the profile picture of the user who created the workflow and assigned it to the current user. The “Description” field displays the full text of the task. The “Reject” button rejects the workflow, removes it from the list on the “Workflows” page and notifies the assigner. The “Start” button at the bottom of the page lets the user confirm that they have started working on the workflow and the system notifies the assigner about it. The “Complete” button marks the workflow as completed, moves it to the “Completed” list on the “Workflows” page and notifies the assigner.

8. Wireframe of viewing files



Explanation: The “Files” page provides functionality for the user to view the list of files available to them and to manage them. The upper side panel contains the breadcrumb navigation and the “Search” field, which accepts user input and searches for files. The middle panel displays the list of files.

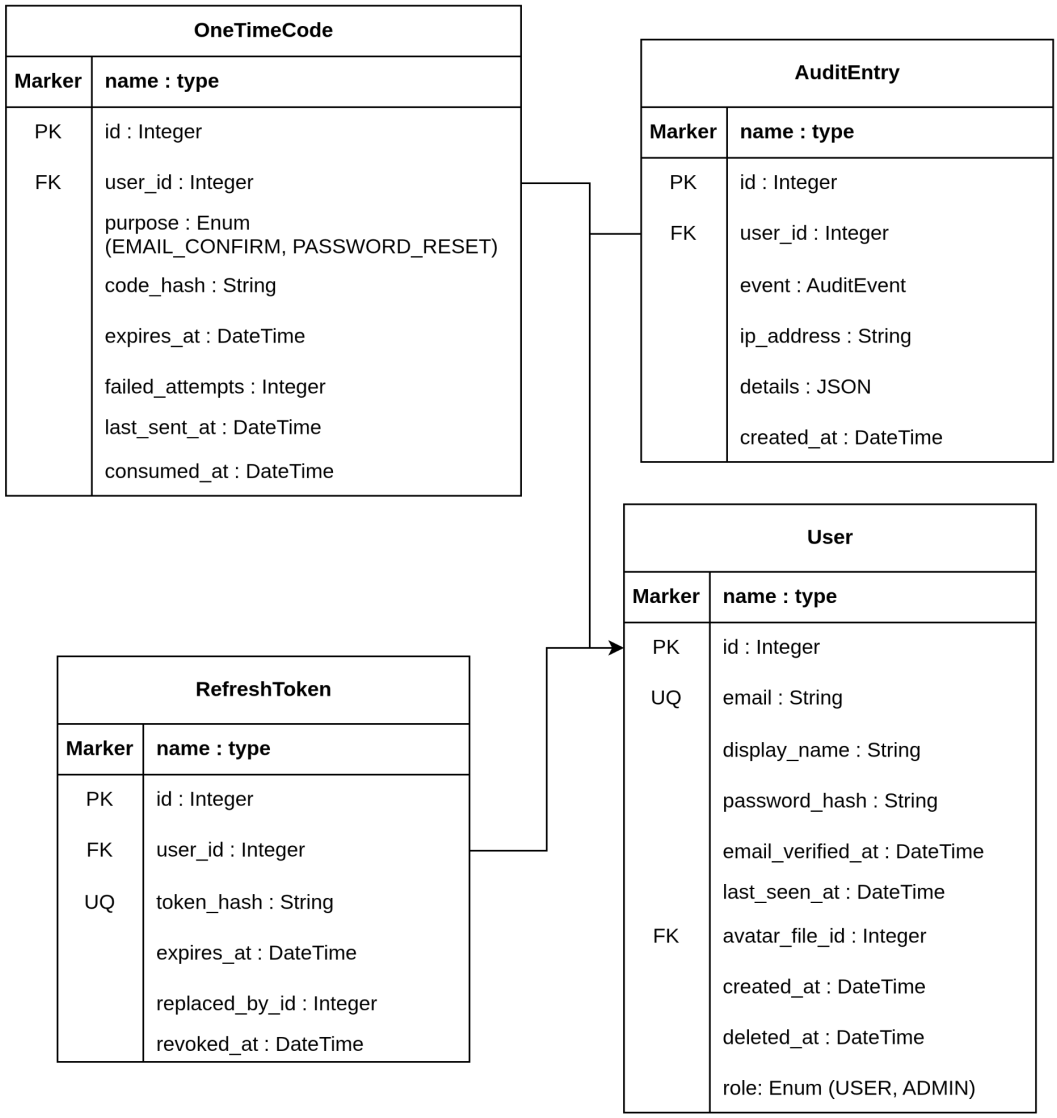
The “Received” and “Shared” buttons above the list filter the files by their origin. The buttons in the middle panel let the user click on a file and open a panel next to it. The context menu contains the “Open”, “Download”, “Print” and “Share” buttons.

7. Data model

PR - primary key, FK - foreign key, UQ - unique.

This section describes the persistent business entities. Each box is one entity, with fields written as ‘name : type’. Markers: PK = primary key, FK = foreign key, UQ = unique. Lines are associations. Links to User through fields such as creator_id or uploader_id are not drawn, to keep the diagrams readable; the FK marker shows them. Entities are grouped by bounded context (5.3). References between contexts are by ID only; the implementer decides whether to enforce them as database foreign keys, but a context must never read another context’s tables (MAIN-01). All IDs are server-generated integers (auto-incrementing) and all timestamps are UTC (USE-06). Field length limits are in section 4.9. Three things are deliberately not stored as entities: user presence (kept in memory, 5.5.1), WebSocket connections, and the search index (derived from messages and file names).

7.1. User, OneTimeCode, RefreshToken, AuditEntry



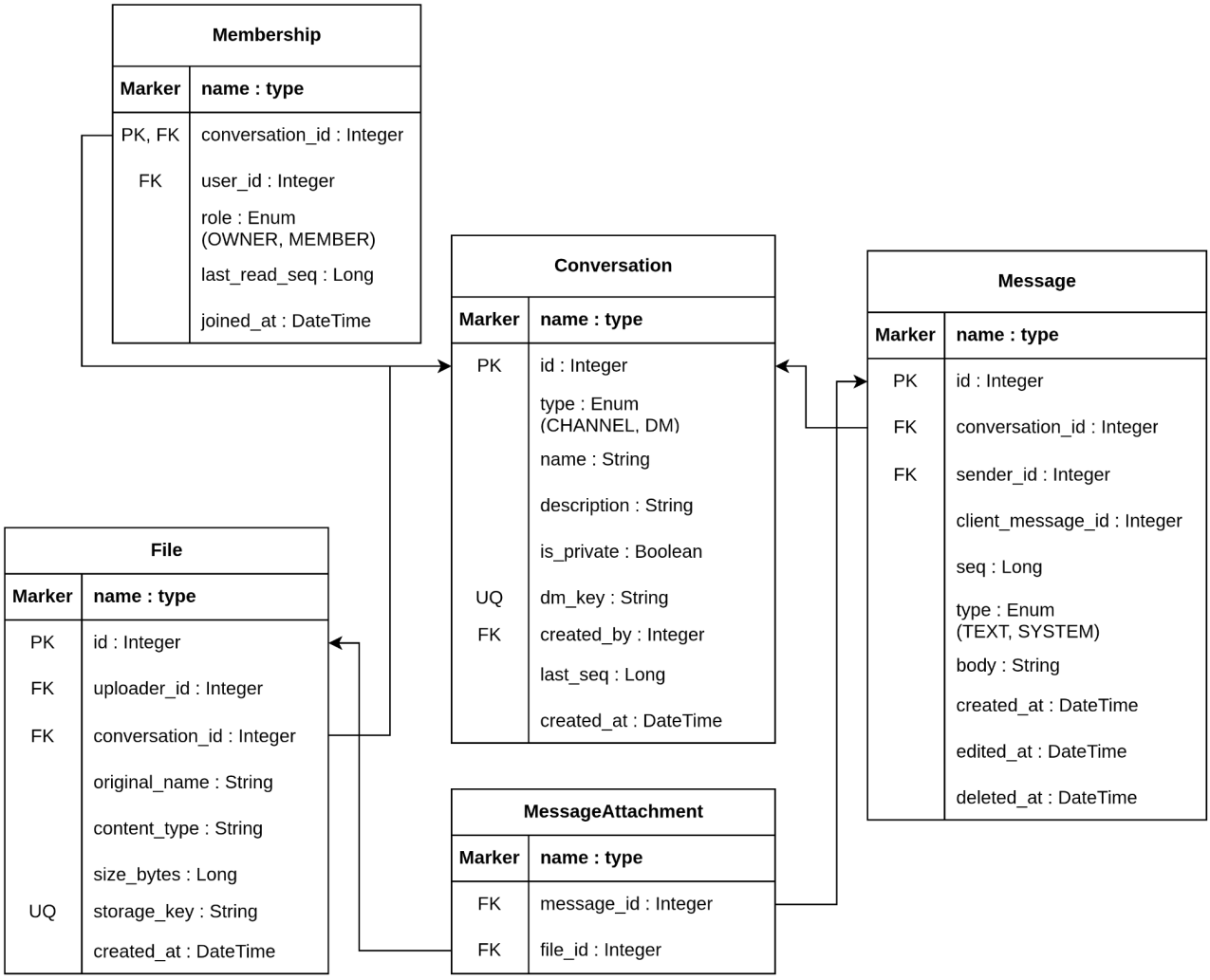
User is a registered person. A user is created at registration, but it counts as active only when email_verified_at is set. last_seen_at is updated when the last connection closes. On account deletion (SEC-10), email, display_name and password_hash are replaced by empty or placeholder values and deleted_at is set; the row is kept so old messages show “Deleted user”. avatar_file_id references a File whose conversation_id is empty. Roles are either USER or ADMIN.

OneTimeCode is a 6-digit code for email confirmation or password reset (SEC-04). Only the hash is stored. A code is valid when expires_at is in the future, consumed_at is empty and failed_attempts is below 5. last_sent_at enforces the 60-second resend limit.

RefreshToken stores the hash of an issued refresh token (SEC-03). On every refresh the old token is revoked and replaced_by_id points to the new one. A token is usable only if revoked_at

is empty and expires_at has not passed.
AuditEntry records security events (SEC-09). Entries are deleted after 90 days.

7.2. Message, Conversation, Membership, MessageAttachment, File



Conversation is either a channel or a direct message (DM), distinguished by type. Channels have a name (unique, case-insensitive) and an optional description. Private channels (is_private = true) are visible only to members; public channels can be joined by any user. A DM has no name, exactly two members, and a dm_key (the two user IDs sorted and joined with ":"), which is unique so two users can never have two DMs. last_seq is the sequence number of the latest message.
Membership links a user to a conversation. The key is (conversation_id, user_id). role is OWNER (the creator) or MEMBER. last_read_seq stores how far the user has read, so unread

count = Conversation.last_seq − last_read_seq. Leaving a conversation deletes the row.
Message is one entry in a conversation. In a single transaction the server increments Conversation.last_seq and assigns it to seq, so (conversation_id, seq) is unique and defines the order (REL-04). client_message_id is generated by the client; (sender_id, client_message_id) is unique, so a repeated request returns the original message (REL-03). type is TEXT (written by a user) or SYSTEM (generated by the server, with an empty sender_id). A deleted message keeps its row; deleted_at is set, and clients show “message deleted” instead of the body.
MessageAttachment links a message to a file. The key is (message_id, file_id), and a message has at most 10 attachments. The same file can be attached to several messages, which is how “Share” works.
File holds metadata only; the content is in file storage under the random storage_key (SEC-08). content_type is determined from the file content, and size_bytes is at most 25 MB. conversation_id is the conversation where the file was first uploaded; it is empty for avatars. A user may access a file if they are a member of any conversation that has a message with this file attached (SEC-05).

7.3. Workflow, Notification

Workflow	
Marker	**name : type**
PK	id : Integer
FK	conversation_id : Integer
FK	creator_id : Integer
FK	assignee_id : Integer
	title : String
	category : String
	description : String
	importance : Enum (ASAP, SOON, EVENTUALLY)
	status : Enum (OPEN, IN_PROGRESS, COMPLETED, REJECTED)
	created_at : DateTime
	updated_at : DateTime
	completed_at : DateTime

Notification	
Marker	**name : type**
PK	id : Integer
FK	recipient_id : Integer
	type : Enum (MESSAGE_SENT, MEMBER_JOINED, WORKFLOW_ASSIGNED, WORKFLOW_STATUS_CHANGED, WORKFLOW_REJECTED)
	conversation_id : Integer
	message_id : Integer
	workflow_id : Integer
	text : String
	read_at : DateTime
	created_at : DateTime

Workflow is a task attached to a channel (not to a DM). creator_id is shown as “Assigner” in the UI, and assignee_id must be a member of the channel. status follows the state machine in 5.5.2: OPEN → IN_PROGRESS when the assignee presses “On it!”, and IN_PROGRESS → COMPLETED when the assignee presses “Completed”, or REJECTED when it is rejected. No other transitions are allowed, and only the assignee may change the status (others get an error). completed_at is set on completion. importance (ASAP, SOON, EVENTUALLY) is a plain attribute and doesn’t affect the state. Creating a workflow publishes WorkflowAssigned; each status change publishes WorkflowStatusChanged; rejecting a workflow publishes WorkflowRejected) In the “Workflows” screen, the tabs Due and Completed filter by status, and ASAP / Soon / Eventually filter by importance.

Notification is one item in a user’s “Latest notifications” list. It is created for each recipient when a subscribed event occurs (5.3). text is a ready-to-show summary. The link fields (conversation_id, message_id, workflow_id) tell the client which page to open on click; only the ones relevant to the type are set. read_at is empty until the user opens it. Notifications are deleted after 90 days.

8. API contract

The complete and executable REST and WebSocket API contract for TeamChat is defined in the OpenAPI 3.0 specification file (*openapi.yaml*), which is included as an attachment to this project documentation.

8.1 Overview & Conventions

Base Path & Protocol: All REST endpoints operate under the base URL *https://api.teamchat.internal/api/v1* and require HTTPS (SEC-02).
Authentication: Protected endpoints require an Authorization: Bearer <access_token> header. Access tokens have a maximum lifetime of 15 minutes, while refresh tokens last for 7 days and are rotated on every use (SEC-03).
Data Standards: All request and response bodies use JSON with snake_case field naming. Identifiers are integers (auto-incrementing), and timestamps follow ISO 8601 UTC strings (YYYY-MM-DDTHH:MM:SSZ), satisfying USE-06.
Error Handling: Non-2xx responses return a standardized JSON structure (ErrorResponse) containing an error code, human-readable message, and optional field-level validation details (USE-03).
Real-Time Protocol: Real-time event streaming and presence updates are handled via a single WebSocket connection (/ws), authenticated via a short-lived single-use ticket (SEC-03).

8.2 Structure of the Attached Contract (openapi.yaml)

The attached OpenAPI file covers the complete system architecture and maps directly to the backend bounded contexts (Section 5.3):

- **Auth**: Account registration via 6-digit email confirmation codes, login, token refresh, password recovery, and session management.
- **Users**: Profile management, user searching, avatar uploads, and GDPR data exports (FR-04, FR-06).
- **Conversations & Messages**: Channel creation, public/private channel browsing, direct messaging, cursor-based message pagination, idempotent message sending (client_message_id), and read receipts (FR-09 through FR-17, REL-03).

- **Files**: Multi-part file uploads (up to 25 MB), metadata management, and secure streaming downloads with proper content-type validation (SEC-08).
- **Workflows**: Task creation within channels, stage transitions ("On It!" and "Completed"), and importance filtering (FR-34 through FR-38).
- **Presence, Notifications**: Manual and automatic presence tracking, unread notifications list (FR-32,FR-33, FR-39, FR-40).
- **Realtime (/ws)**: WebSocket server event envelopes (WsServerEvent) and client actions (WsClientEvent) for instant event propagation.